Cluster Computing (2026) 29:837
https://doi.org/10.1007/s10586-026-06522-x

Towards automated formal verification of authorization policy in microservice systems

Connor Wojtak[1] · Shakthi Weerasinghe[1] · Nilesh Rao[1] · Tomas Cerny[1]

Received: 30 October 2025 / Revised: 9 July 2026 / Accepted: 10 August 2026
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract
When implementing authorization policies across microservices, their individual development and decentralized evolution introduce consistency errors that slip through static analysis and compartmentalized testing. They manifest as runtime security vulnerabilities leaving no trace until exploited. Identifying such authorization consistency errors demands complex in-depth analysis of interservice communication pathways and intra-service call graphs, which is infeasible to perform manually at scale. Existing approaches lack the ability to automatically verify end-to-end authorization consistency across distributed service boundaries. To guarantee consistent authorization policy while reducing developer burden, we apply a formal method-based approach that provides an automated solution. Our methodology encodes microservice system source code into a system constraint model that enables fully automated formal verification of authorization policy across the entire system without requiring manual specification of system behaviors. Formal reasoning and proofs demonstrate the correctness of the methodology. A proof of concept is developed and applied in a case study, which shows high precision and computational performance in identifying authorization policy inconsistencies. Our formal proofs and case study establish the correctness and practical applicability of our methodology for identifying and addressing consistency errors.

Keywords Verification · Static analysis · Microservices · Authorization · SMT solving · Maintainability · Decentralization

1 Introduction

Ensuring correct implementation of application-level authorization policy in software systems is critical to the confidentiality, integrity, and availability of system resources and data [1]. Yet, despite the mass adoption of microservice architecture across various industries and their sectors, ensuring enforcement of authorization policies in these systems remains challenging [2].

Difficulties emerge for two main reasons. First, the system code is decentralized into self-contained microservice codebases often managed by independent teams [3]. As the microservices evolve with respect to market demands and functional adaptation [4], developers may overlook the implications that code changes have on the correctness of microservice authorization policies. Microservice developers are not security experts and typically operate in a *bounded codebase* (i.e., within only a few microservices [3]), and thus they **lack the complete system perspective** in code reviews or trade off reasoning [5]. Second, since there is no global perspective to evaluate the overall microservice system statically, practitioners resort to runtime testing [6, 7]. This approach faces two critical limitations: (1) maintaining comprehensive and up-to-date test suites, and (2) executing the entire test suite for each pull request to the code repository, which demands considerable computation power [8].

✉ Connor Wojtak
connorwojtak@arizona.edu

Shakthi Weerasinghe
syweerasinghe@arizona.edu

Nilesh Rao
nilesh@arizona.edu

Tomas Cerny
tcerny@arizona.edu

[1] Department of Electrical and Computer Engineering, University of Arizona, 1230 E Speedway Blvd, Tucson, Arizona 85719, USA

Published online: 12 October 2026

This lack of a centralized view and difficulty in verification means that one microservice may implement an authorization policy that is more permissive or restrictive than another microservice expects. These differences between the policy a microservice implements and the policy expected by another microservice are **inconsistencies**, which introduce *unintended authorization faults*, such as how T-Mobile's January 2023 incident mis-implemented policy exposed an endpoint for querying confidential customer data, eventually costing them at least approximately $30 million [9]. However, these errors are not immediately obvious. These errors slip through static analysis and compartmentalized testing, manifesting only at runtime as security vulnerabilities that enable unauthorized data access and privilege escalation. Authorization policy inconsistencies occur at multiple layers in a microservice system. Not only do endpoints need to have policies properly applied, but the indirect effect of these policies on the data these endpoints access must be considered as well. Analyzing the authorization policy at multiple layers is inherently complex, highlighting the need for an automated and efficient solution to ensure microservice system authorization policy consistency.

For example, consider when a user invokes an endpoint within a microservice system to get details about a book (Fig. 1). The user might be authorized to call this endpoint. However, a single service call may trigger a chain of dependent calls to multiple downstream services as a result of service choreography [10]. A request for a book may also fetch metadata about a book's author from another service. An authorization inconsistency may occur when, due to some policy misconfiguration, a user authorized for the initial request is denied access at some point in the resulting call chain. This problem is further complicated when the authorization policy for access to data entities is inconsistent.

Current error mitigation practices are difficult to comprehensively maintain [6]. Code reviews are limited to a single codebase due to the lack of a central system view [3], failing to consider the implemented authorization policy in the context of the other microservices. Moreover, tests fall short of identifying all possible errors, given the system complexity and the combination of various communication paths, layers, and authorization roles. As a result, a faulty system version might be deployed to the production environment.

By analyzing the authorization policy implemented by each microservice, comparisons can be made between each policy implementation to identify these inconsistencies. This paper applies formal methods to identify inconsistencies to address the challenge of identifying authorization faults in microservices using a more comprehensive and efficient method than current testing or analysis safeguards. It derives a constraint-aware Intermediate Model (IM) and uses a solver-based system to ensure the consistency of access permissions across the entire system. We create constraints to identify inconsistencies at two layers — the endpoint layer and the data layer.

Our preliminary work [11, 12], which we extend here, applies formal methods to verify cross-cutting concerns such as authorization policy. This manuscript delivers a comprehensive perspective, bringing the following novel contributions:

- A formally defined constraint system for verifying various inconsistencies in authorization policy between microservices at the endpoint and data entity layers.
- Formal reasoning about the feasibility and correctness of our methodology for verifying authorization policy.
- A prototype implementation of the methodology.
- A fully detailed case study on a mid-sized microservice system benchmark to illustrate the applicability of our methodology to realistic systems.

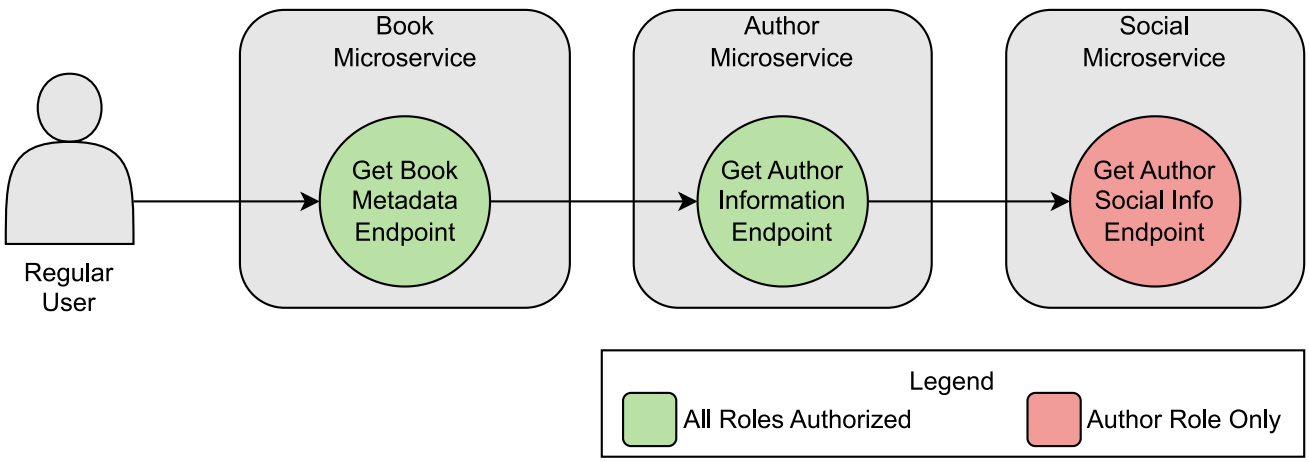


Fig. 1 A visual illustration of an endpoint call flow to get information about a book. The transaction fails for the regular user with the call to the author's social information

- A reproduction package to facilitate the replication of our results.

This paper is organized as follows. Section 2 elaborates on the problem's background and related work. Section 3 identifies our methodology for formally verifying microservice authorization policy. Section 4 presents formal reasoning and proofs to support our methodology. Section 5 illustrates the results of our case study. Section 6 addresses potential threats to the validity of our methodology. Section 7 closes with a discussion of the impacts of our work and future directions.

2 Background and related work

For software verification, there is a choice between static and dynamic analysis. For example, dynamic analysis and runtime testing can be used for software verification of various concerns including authorization policy. However, in microservice systems, each microservice implements its own authorization enforcement [2]. Since microservices may call each other [10], this means that end-to-end testing of entire workflows must be performed to verify authorization policy. A systematic mapping study by Hui et al. [13] reveals a significant trade-off between cost and test suite comprehensiveness, a sentiment echoed by Smith et al. [7]. Thus, while static analysis does not capture the same behavioral semantics and execution that runtime testing provides, static analysis of a system model offers a more cost-effective alternative. Further, for properties that can be extracted from static analysis (e.g., potential control flows as opposed to measuring execution time), static analysis allows for systematic verification across the entire system. This potentially allows for more comprehensive analysis and code coverage for these properties or concerns than runtime testing could provide. Static analysis and runtime testing could then compliment each other, with static analysis offering quick, comprehensive analysis of some concerns, and runtime testing covering more complex analysis of concerns that cannot be obtained via static analysis.

2.1 System modeling and information extraction

Various approaches have been proposed to extract architectural information about microservice systems, enabling system modeling. [14, p. 4–5] propose dynamic extraction of information from the source code via "concolic execution" which runs the program with symbolic inputs to identify control paths and policy. However, dynamic analysis is costly and difficult to maintain [7], making static analysis the preferred approach. Genfer and Zdun [15] propose a static pattern matching system for identifying and collecting system components, connections, and attributes. Cerny et al. [16] develop a system that uses an abstract syntax tree (AST) to extract system components, connections, and attributes from the code, generating an intermediate representation (IR) of the system. These IRs can be incrementally updated as changes are made to the system code repositories [17].

Identifying system components is critical because, in cloud-native systems, authorization policy enforcement at the application layer is typically performed at the endpoint level [18]. Most or all information about a microservice system and its enforced authorization policy can be extracted statically from the source code [17, 19]. For illustration, Enterprise Java defines standard specifications such as JSR 250 [20] and JSR 375 [21], which provide Java annotations and features that support role-based authentication at the endpoint level, implemented by *Spring Security* [19]. Role-based access control (RBAC) assigns users to roles, which define the permissible operations a user may perform. A robust RBAC mechanism should conform to its principles at the API endpoint level while also accounting for fine-grained access to data entities. RBAC support is widespread across microservice system benchmarks, which streamlines validation [19–23]. Thus, a system model enables static analysis of a microservice system.

2.2 Formal methods for software verification

A significant body of work has performed software verification using system models by applying formal methods. Britikov et al. [24] and Ringer et al. [25] propose methodologies for encoding source code — including function calls, control structure and paths — into Satisfiability Modulo Theories (SMT) constraint models. Such models can be evaluated by a solver (e.g., Z3 [26], cvc5 [27]) and used to generate test cases covering various execution paths. Lattuada et al. [28] develop a methodology that utilizes SMT solvers to eliminate bugs in software at multiple levels, including system and memory levels.

2.3 Authorization policy verification

Several studies apply formal methods specifically to authorization consistency. Cau et al. [29] encode a model of an application and uses binary decision diagrams to verify proper enforcement of authorization policy. Armando and Ranise [30] use formal methods and theorem solving to identify authorization policy consistency regardless of the number of users and roles present in a system. [31] use the Prover9 theorem solver to identify inconsistencies between users, their roles, and their ability to access data records.

However, this work targets monolithic and non-distributed systems.

In the context of cloud-native systems, there are both commercial and academic solutions for verifying authorization policy. Security in cloud-native environments applies at multiple layers, including the application, infrastructure, and network layers. Existing static verification methods focus on the infrastructure and network layers, leaving a critical gap at the application layer.

One example at the infrastructure layer is Amazon Web Services' (AWS) Zelkova system, which uses formal methods to analyze and verify Identity and Access Management (IAM) policies. Such policies are used to define access to certain operations and resources in AWS [32, p. 2]. This system is used in several of their products, including AWS Config, Amazon GuardDuty, and IAM Access Analyzer [33]. Their approach decomposes policy statements into an "abstract syntax" that is then encoded into SMT constraints and used to analyze or verify the policies. While this modeling approach shares similarities with ours, it operates at the infrastructure layer and cannot verify application-level authorization policy across distributed microservices.

Meanwhile, OSO [34] offers authorization-as-a-service for policy declaration and management, utilizing its domain-specific policy specification language [35] to define authorization rules. The policy is consistently enforced by having microservices query OSO cloud servers during runtime to determine if a request is authorized based on the defined policy. This centralized enforcement can help prevent inconsistencies from occurring across microservice systems. However, this approach introduces a single point of failure and a scalability bottleneck. Centralizing authorization policy enforcement is considered an anti-pattern [36], making OSO unsuitable for production systems at scale. Thus, there is still a need for a methodology to ensure authorization consistency in cloud-native systems at the application layer.

In academia, Lee and Winslett [37] propose formal methods for validating credentials at the application layer in distributed systems, identifying cases where credentials could expire mid-transaction without causing the transaction to fail. However, this focuses on authentication rather than authorization.

Abdelfattah et al. [2] propose employing static analysis to enumerate create, read, update, and delete (CRUD) data operations performed in chains of endpoint calls, identifying required permissions through repository accesses within endpoints. However, they do not apply formal methods but instead calculate access rights without verification guarantees. Applying formal methods to our problem domain will enable us to verify policy consistency with mathematical rigor and automatically suggest fixes for identified inconsistencies, rather than just calculate access rights.

2.4 Research gap and our contributions

Table 1 summarizes the capabilities of existing approaches compared to our methodology. Our preliminary work has just only begun to address identified gap. Initial work in [11] envisions formal verification of authorization policy but provides limited empirical analysis and formal reasoning. Meanwhile, our work in [12] demonstrates how a formal model can verify authorization policy for microservices but focuses chiefly on verifying architectural rules rather than authorization policy. This paper builds on our prior work [11, 12] to deliver a complete methodology with formal correctness proofs, a working implementation, and empirical validation. We address the critical gap in application-layer authorization verification for microservice systems, providing an automated, scalable solution that existing approaches does not offer.

3 Methodology

Our methodology for verifying authorization consistency across microservice systems has three stages (Fig. 2): (1) encoding the microservice system source code into an Intermediate Model (IM) based on an *Intermediate Representation* (IR) that captures authorization policies and

Table 1 Comparison of authorization verification approaches, whether they discuss formal methods (FM), distributed systems (DS), cross-service verification (CSV), scalability in runtime testing (SRT), and automation (Auto)

Approach	Layer	Analysis type	FM	DS	CSV	SRT	Auto
Lattuada et al. [28]	Infrastructure	Static	✓	×	×	×	×
Cau et al. [29]	Application	Static	✓	×	×	×	✓
Armando and Ranise [30]	Application	Static	✓	×	×	✓	✓
Sabri [31]	Application	Static	✓	×	×	×	✓
Backes et al. [32]	Infrastructure	Static	✓	✓	×	✓	✓
OSO [34]	Application	Dynamic	×	✓	✓	×	×
Abdelfattah et al. [2]	Application	Static	×	✓	✓	✓	✓
Our approach	**Application**	**Static**	✓	✓	✓	✓	✓

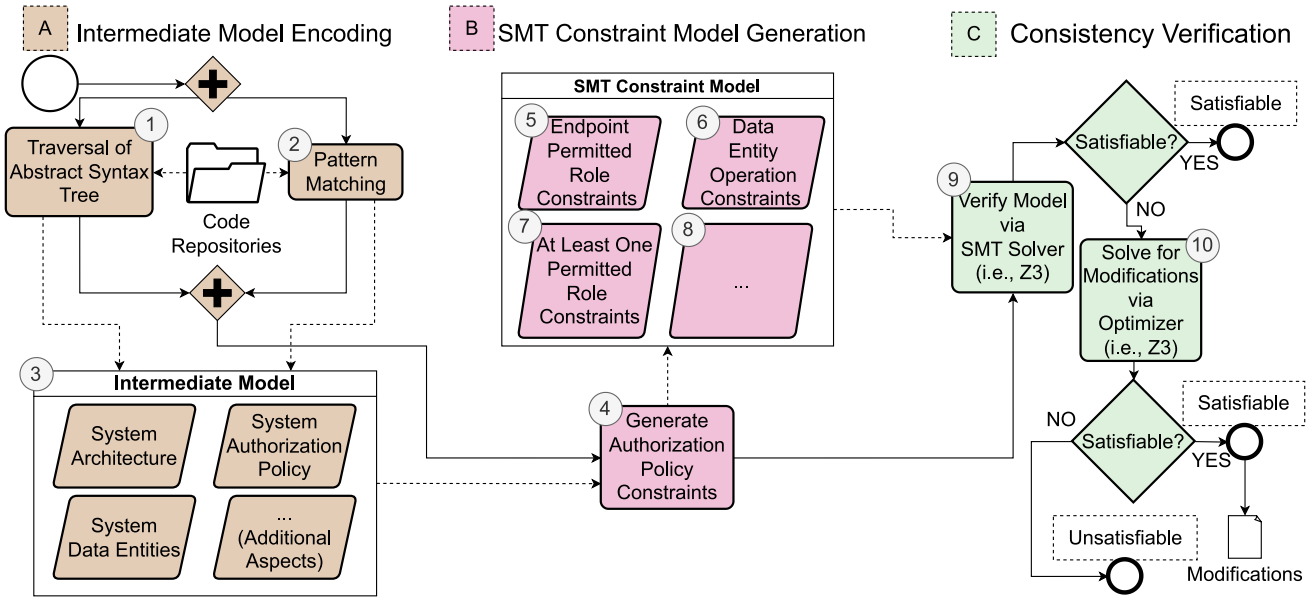


Fig. 2 An overview of the authorization consistency verification process, starting with the encoding of the IM and finishing with the application of an SMT solver to verify authorization policy consistency

service interactions, (2) generating a comprehensive SMT constraint model from this IM, and (3) applying an SMT solver to verify consistency and suggest corrections when inconsistencies are detected. We detail the IM generation process in Sect. 3.1.

After encoding the microservice system into an IM, an SMT constraint model can be generated. We identify three types of constraints for detecting inconsistencies in the authorization policy — (1) endpoint permitted role consistency constraints (Sect. 3.2.1), (2) data entity operation consistency constraints (Sect. 3.2.2), and (3) at least one permitted role constraints (Sect. 3.2.3). By traversing the IM and considering each role, endpoint, and data entity, a comprehensive set of SMT constraints that cover the entire microservice system can be generated.

Once the SMT constraint model is complete, the verification of the system is performed by providing the constraint model to an SMT solver. The solver's result indicates whether the microservice system contains policy inconsistencies or not. When inconsistencies are detected, the solver can generate a set of changes to restore policy consistency if such a set of changes exists. We detail this process in Sect. 3.3.

Throughout this section, we will refer to a running example of a Library Management System (LibMS). This system consists of two primary microservices: a `CatalogService` and a `LoanService`. The authorization policy defines two user roles: `User` and `Librarian`. The `CatalogService` exposes endpoints such as `GET /books` (accessible by both roles to read `Book` data entities) and `POST /books` (restricted to the `Librarian` to create `Book` entities). Meanwhile, the `LoanService` exposes a `POST /loans` endpoint (accessible by the `User` role) that creates a `Loan` entity and makes a remote service call to the `CatalogService` to verify book availability. We use this system to ground our formal definitions and constraint generation.

3.1 Encoding a microservice system into an intermediate model for SMT solving

To enable formal verification, we must extract and encode the authorization policy and architectural structure of the microservice system. This encoding process transforms the distributed codebase into a unified formal representation suitable for constraint-based analysis.We first define the elements of the model in Sect. 3.1.1, explain how an AST or similar static analysis can be used to extract the elements fo the model in Sect. 3.1.2, and identify how the elements of the model can be encoded in an SMT-compatible way in Sect. 3.1.3.

3.1.1 Model definition

The IM captures all relevant aspects of the microservice system necessary for authorization verification. The IM differs from an IR in that an IM synthesizes both architectural and authorization policy information. Figure 3 illustrates the information we collect and encode to prepare the IM. The microservice system consists of a set of interconnected microservices, each with their own sets of endpoints.

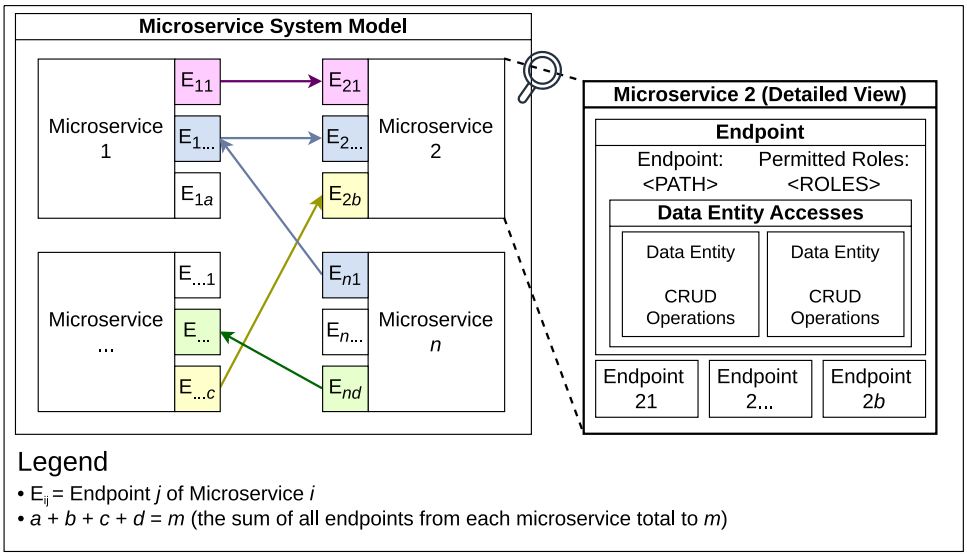


Fig. 3 A visual representation of the IM. Arrows between endpoints indicate system dependencies. Colored endpoints and arrows indicate endpoint call chains

We extend the microservice system model introduced in our prior work [12]. To keep this paper self-contained and to support the formal constraints and proofs developed here, we reproduce below the definition of the extended authorization policy model $\mathcal{S}_{authorization}$:

$$\mathcal{S}_{\text{authorization}} = (\mathcal{M},\ \mathcal{E},\ \texttt{E_parent},\ \mathcal{G},\ \mathcal{D},\ \mathcal{A},\ \mathcal{O},\ \texttt{A_operations},\ \mathcal{R},\ \texttt{E_permitted},\ \texttt{E_accesses},\ldots)$$

where:

Symbol	Definition
$\mathcal{M} = \{M_1, \ldots, M_n\}$	Set of microservices.
$\mathcal{E} = \{E_1, \ldots, E_m\}$	Set of all endpoints; many endpoints belong to a single microservice.
$\texttt{E_parent} : \mathcal{E} \to \mathcal{M}$	Maps an endpoint to its parent microservice.
$\mathcal{G} \subseteq \mathcal{E} \times \mathcal{E}$	Directed graph representing remote service calls between endpoints.
$\mathcal{D} = \{D_1, \ldots, D_h\}$	Set of data entities.
$\mathcal{A} = \{A_1, \ldots, A_j\}$	Set of data entity accesses for entities in $\mathcal{D}$ (create, read, etc.), where each endpoint in $\mathcal{E}$ has at most one access per data entity.
$\mathcal{O} = \{O_1, \ldots, O_k\}$	Set of possible operations on data entities (typically: create, read, update, delete).
$\texttt{A_operations}\mathcal{A} \to \mathcal{P}(\mathcal{O})$	Maps a data entity access to the set of operations it performs, where $\mathcal{P}(\mathcal{O})$ denotes the power set of $\mathcal{O}$.
$\mathcal{R} = \{R_1, \ldots, R_\ell\}$	Set of possible user roles in the microservice system.
$\texttt{E_permitted}\ \mathcal{E} \to \mathcal{P}(\mathcal{R})$	Maps an endpoint to the set of roles permitted to access it, where $\mathcal{P}(\mathcal{R})$ is the power set of $\mathcal{R}$.
$\texttt{E_accesses}\ (\mathcal{E}, \mathcal{D}) \to \mathcal{A}$	Maps an endpoint/data entity pair to a single data entity access.

Illustratively, consider the LibMS and how it would be represented as an IM. Here, $\mathcal{M} = \{\texttt{CatalogService}, \texttt{LoanService}\}$, $\mathcal{R} = \{\texttt{User}, \texttt{Librarian}\}$, and $\mathcal{O} = \{create, read, update, delete\}$. The mapping $\texttt{E_parent(POST /books)} = \texttt{CatalogService}$. The remote call from the loan service to the catalog service is represented in our directed graph as $(\texttt{POST /loans}, \texttt{GET /books}) \in \mathcal{G}$. $\mathcal{D}$ could contain entities such as `Book` or `Loan`. $\texttt{E_accesses(POST /books, Books)}$ would represent an access of the `Books` data repository. Such an access A would have $\texttt{A_operations}(A) = \{create\}$ since `POST /books` creates new `Book` entities. Since `GET /books` allows both `User` and `Librarian`, $\texttt{E_permitted(GET /books)} = \{\texttt{User}, \texttt{Librarian}\}$.

By extracting the microservices, their endpoints, their interactions with data entities, the related authorization policy, and interservice communications, we can effectively analyze distributed authorization policy at multiple layers. The model $\mathcal{S}_{authorization}$ is representative of the IM at ? in Fig. 2.

We note one complexity. Each microservice should own its data, meaning it is sometimes necessary to replicate data between microservices across various bounded contexts. This raises the question: should replicated data in different microservices be considered distinct or equivalent for the purpose of data entity operation consistency?

Treating them as distinct allows different permissions per role across services, reflecting the idea that similar data may serve different purposes depending on context. Conversely, treating them equivalently would require enforcing uniform access controls across services, ensuring consistent system-wide data access.

While both approaches are valid, our methodology adopts the approach that data entities are equivalent across

microservices to enforce consistency in access control across the system. However, the choice of approach could depend on an organization's goals and policies.

To encode the microservice system's architecture and policy into an IM from its code repositories, we use both pattern matching and an IR derived from an AST (see ? and ? in Fig. 2) We iterate through the IR to encode microservices, endpoints, data entities, and connections in between all the endpoints and entities. Some information — namely, the endpoint security configuration — may not present in an IR but present in the source code. Hence, regular expressions can be applied to extract the relevant security configuration. By matching on function names, annotations, or other various code or policy definition aspects, the enforced policy and its scope can be extracted and encoded.

While pattern matching could potentially be used to generate the entire IM, an IR is optimal for this use case for two reasons. First, the IR is derived solely from the source code of the microservice system. Second, it allows for the static extraction of endpoint and function calls across the entire system.

However, the source code may not always state or enforce authorization policies. When there are no defined authorization requirements for an endpoint, we assume that any role can access that endpoint.

3.1.2 Model extraction

The model $\mathcal{S}_{authorization}$ can be extracted via various static analysis techniques, including a combination of analyzing ASTs or IRs and pattern matching and regular expressions. The general flow is to identify each of the elements of the microservice system (e.g., its microservices and then its endpoints) before identifying their properties and the connections between them (e.g., remote service calls and access properties).

The microservices can be identified based on project structure, including folder structure or key project file definitions (e.g., a pom.xml file or a pyproject.toml). An IR or AST can be analyzed for each microservice. Additional analysis of IRs or ASTs for entrypoints by searching through the application's function definition can also be used to confirm microservices.

The endpoints can be identified by iterating through the IR or AST for a microservice and searching for functions with specific decorators, annotations, or other properties attached to them in the AST/IR identifying them as endpoints. The mapping of E_{parent} is trivial as the definition of the endpoint would appear only in the AST/IR for the microservice the endpoint is a part of.

Identifying remote service calls to populate $\mathcal{G}$ requires a breadth-first or depth-first search of all the function calls made in an endpoint. For example, starting at a function that serves an endpoint, recursively search all of the calls that function makes, along with the calls those function calls make, etc. A remote service call can then be found by looking for known library functions that actually perform the call. The remote service call URL can be extracted from the function call parameters, allowing for the tracing of the original endpoint function to the endpoint represented by the URL. This forms the connections in $\mathcal{G}$, that, when all connections have been extracted, allow for the tracing of remote service call chains from endpoint to endpoint.

Meanwhile, data entities can be extracted by iterating through AST/IRs and identifying data models by various decorators, annotations, or other properties attached to them in the AST/IR identifying them as data entities.

Access to data entities (where an endpoint performs an operation on a data entity) can be identified similar to how remote service calls are performed, allowing for $\mathcal{A}$ and `E_accesses` to be constructed. A breadth-first search or depth-first search can iterate through all the function calls an endpoint makes to identify database function calls that act on the data entity.

The set of possible data operations $\mathcal{O}$ is not derived from the system but rather is kept constant between systems. This set typically contains "create", "read", "update", and "delete" but can be re-configured to represent different sets of operations since all operations in $\mathcal{A}$ are mapped to some subset of $\mathcal{O}$ by `A_operations`. The type of operation being performed can be derived from the name of the database function call. Thus, as long as the types of operations are derivable from the name of the database function call, different types of operations can be included in $\mathcal{O}$.

The set of possible user roles in the system can be performed by identifying security configuration by iterating through classes or functions in the AST/IR and searching for various decorators, annotations, etc. Then, analysis of security configuration through pattern matching can be used to identify all the roles in the system. Additionally, analyzing the security configuration for what roles are permitted for what functions allows for the construction of `E_permitted`.

While implementation may vary based on the framework or language, this methodology generally allows for the construction of the model for any programming language from which an AST (and thus IR) can be derived.

3.1.3 Model representation

From the formal definition of the model $\mathcal{S}_{authorization}$, a method is necessary to encode this model into a solver-friendly format that can be used to generate the SMT constraint model. We provide the following solver-friendly representations of $\mathcal{S}_{authorization}$:

$\mathcal{M}$	can be represented as a set M where $M_i \in M$ is a subset of distinct endpoints from $\mathcal{E}$ where $i \in [0, n-1]$.
$\mathcal{E}$	can be represented as a set E where each $E_i \in E = i$ where i is an integer and $i \in [0, m-1]$.
`E_parent`	can be represented as an array of integers $E_{parents}$ where each element $E_{parents_i}$ is an integer $j \in [0, n-1]$ such that $E_{parents}(i) = j$, where $i \in [0, m-1]$ corresponds to an endpoint in $\mathcal{E}$ and j corresponds to the endpoint's parent microservice in $\mathcal{M}$.
$\mathcal{G}$	can be represented as an adjacency list or matrix G such that $G[i][j] = True$ iff there is an edge from E_i to E_j for $i \in \mathcal{E}, j \in \mathcal{E}$. Otherwise $G[i][j] = False$.
$\mathcal{D}$	can be represented as a set D where each $D_i \in D$ is represented as integer $i \in [0, h-1]$.
$\mathcal{A}$	and `E_accesses`: can be jointly represented as a 2D array A where each $A[i][j] \in \mathcal{Z}+$, i corresponds to the endpoint $E_i \in \mathcal{E}$, and j corresponds to the data entity $D_j \in \mathcal{D}$. Further, each $A[i][j]$ is a bitstring representing `A_operations` such that E_i performs $A[i][j]$ operations on D_j. $A[i][j] = 0$ indicates no operations were performed.
$\mathcal{O}$	|can be represented as a bitstring O, one for each possible operation considered (i.e., create, read, update, and delete).
$\mathcal{R}$	can be represented as a bitstring R, one for each possible user role in the system (i.e., generic user, admin, etc.).
`E_permitted`	can be represented as an array of bitstrings EP in the same form as $\mathcal{R}$'s representation where each element EP_i represents the roles permitted to access E_i.

To illustrate the SMT representation, let us re-consider the LibMS. If $\mathcal{R}$ is represented by a 2-bit string where the first bit is `User` and the second is `Librarian`, the permitted roles for `POST /books` would be encoded in EP as $0b01$, while `GET /books` would be $0b11$. For data operations, for each of the four operation types (*create*, *read*, *update*, *delete*), that *create* maps to $0b1000$, *read* maps to $0b0100$, and so forth. Meanwhile, say that `POST /books` creates `Book` data entities, and that each endpoint is assigned a unique integer i and each data entity is assigned a unique integer j. Then, if 1 is `POST /books`'s unique integer and 3 is `Book`'s unique integer, then $A[1][3] = 0b1000$ represents how `POST /books` creates `Book` entities.

3.2 SMT constraint model generation

After encoding the microservice system into an IM, an SMT constraint model can be generated (see ? in Fig. 2). Eq. 1 illustrates how the constraint model is formed by combining all the endpoint permitted role constraints $C_{endpoints}$ (see ? in Fig. 2), data entity operation constraints $C_{entities}$ (see ? in Fig. 2), and at least one permitted role constraints C_{role} (see ? in Fig. 2). The constraint model is extensible such that new constraints based on the IM could be added (? in Figure 2).

$$C = C_{endpoints} \wedge C_{entities} \wedge C_{roles} \wedge ... \quad (1)$$

When generating the constraints, we hold all variables in the system constant besides the roles authorized to access an endpoint. This allows the solver to solve for a set of authorized roles for each endpoint that makes the system authorization policy consistent. We outline the types of constraints we generate over which domains of system aspects.

3.2.1 Endpoint permitted role constraints

Endpoint permitted role constraints focus on ensuring that, if a role is permitted to access an endpoint $E_{initial}$, it is also permitted to access all of the endpoints that are called as a result of a call to E_{inital}.

The motivation behind enforcing this type of constraint is to identify endpoints that allow certain roles which invoke endpoints that do not allow these roles. Such issues can arise as microservice systems evolve [4], functionality is added/moved/removed, user permissions change, and complex service interactions and remote service calls which difficult to verify [10] are modified. Identifying and addressing these issues can help identify endpoints in service call chains that are too permissive so they can be modified to prevent unwanted roles from performing data operations through said endpoints. Further, these constraints are crucial for the methodology to suggest changes to permitted roles as it ensures any permitted role changes cascade through remote service call chains as needed.

Figure 4 illustrates a sample scenario in the LibMS; users call endpoints $(E_1, ..., E_6)$, which may make calls to other endpoints or perform operations on data entities $(D_1, ..., D_4)$. Consider a scenario where a user calls E_6, a GET book loans endpoint. E_6 makes a remote service call to E_3 to fetch book details for the loan ?, which in turn makes a remote service call to E_2 to fetch author details for the book ?. These successive linked calls from one endpoint to another form endpoint call chains. Along an endpoint call chain, it is crucial

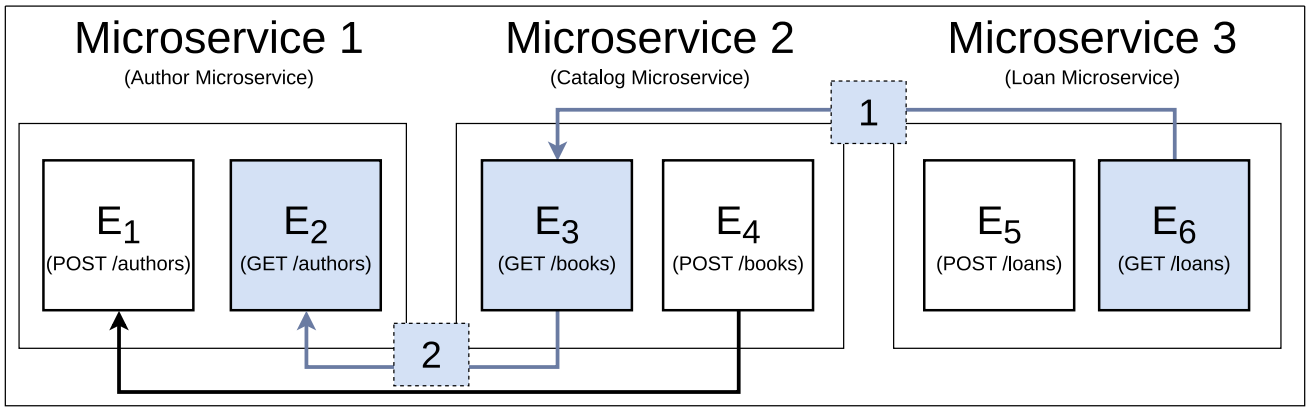


Fig. 4 An example microservice system with interconnections between endpoints displayed

that the user making a call to the first endpoint in the chain is permitted to call all endpoints in the chain. For example, say a user with the generic `User` role is permitted to call E_6. However, if only the `Librarian` role is permitted to call E_3 or E_2, the call to E_6 cannot be successfully completed, as the generic user cannot complete some of the requisite calls. This indicates an inconsistency in the authorization policy. Either the role should be permitted to call all the endpoints in every endpoint call chain originating from the initial endpoint, or the role should not be permitted to call the initial endpoint.

We can formally categorize this type of authorization policy issue. Whenever there is an endpoint $E_i \in \mathcal{E}$ that calls an endpoint $E_j \in \mathcal{E}$, if E_i permits roles that are not permitted by E_j, there is an endpoint permitted role inconsistency. This leads to the formal definition of an endpoint permitted role inconsistency:

Definition 1 *(Endpoint Permitted Role Inconsistencies).* All endpoint permitted role inconsistencies in $\mathcal{S}_{authorization}$ are contained by the set:

$$\mathcal{I}_{endpoints} = \{(E_i, E_j) \in \mathcal{G} \mid \texttt{E_permitted}(E_i) \not\subseteq \texttt{E_permitted}(E_j)\}.$$

The goal is to ensure $\mathcal{I}_{endpoints} = \emptyset$. Thus, we define a constraint to identify the inconsistencies represented in this set. If an endpoint $E_i \in \mathcal{E}$ has a bitstring of allowed roles that permits role $u \in \mathcal{R}$, then all subsequent call chains originating from E_i should include only endpoints that also permit u. Thus, we assert that, for every edge $(E_i, E_j) \in \mathcal{G}$ that all roles authorized to call E_i are also authorized to call E_j. In other words, this means we assert $\texttt{E_permitted}(E_i) \subseteq \texttt{E_permitted}(E_j)$. This forms the basis for Eq. 2 below.

$$C_{endpoints} = \bigwedge_{(E_i,E_j)\in G} (EP[E_i] \,\&\, EP[E_j] = EP[E_i]) \tag{2}$$

We note that this constraint is designed specifically for systems that rely on authorization context propagation without trust boundaries and elevation of privileges via these trust boundaries [38] as this constraint would yield false positives during intended privilege escalation. For example, consider a case if the LibMS example's `LoanService` only allowed `Librarians` access to any of the endpoints, and there was an implicit trust boundary between the `CatalogService` and `LoanService` via architectural configuration, microservice token exchange, etc. This means that if a regular `User` calls a `CatalogService` endpoint which calls a `LoanService` endpoint via an escalation of privileges due to the implicit trust boundary, $C_{endpoints}$ would not be satisfiable.

3.2.2 Data entity operation constraints

Data entity operation constraints prevent one endpoint from allowing a role to perform certain data operations on a data entity when another endpoint prevents that same role from performing those data operations on the same data entity type.

The motivation behind this type of constraint is to identify instances where the data operations an endpoint allows certain roles to perform is too permissive relative to other endpoints. Such issues can arise as microservice systems evolve [4] and users need to perform new operations or stop performing existing operations. These issues can be difficult to identify across distributed data operations and independently implemented services [3, 10]. This can help prevent unintended data leakage or loss of data integrity by identifying instances where a role which shouldn't have access to certain data is provided unintended access via an endpoint.

Consider Fig. 5 which illustrates two LibMS endpoints, E_1 and E_2, in the same microservice for users to rate books, which access data entity D_1 via a data repository. E_1 performs the same types of CRUD operations and more than E_2

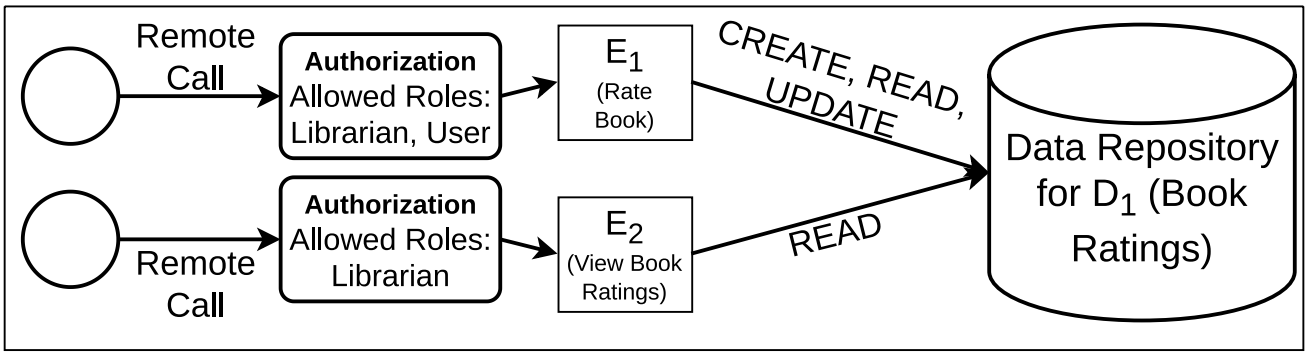


Fig. 5 An example of a data entity operation inconsistency. E_1 and E_2 are endpoints, while D_1 is a data entity accessed via a data repository

does on D_1. *This reveals an inconsistency*. If a generic `User` role can call E_1 to create, read, and update `ratings` (D_1), it is unclear why the same role may not be able to access E_2 to perform only a subset of those operations. A regular user likely should be able to read a book rating if they are able to create, read, and update book ratings in another endpoint. This means either users with the generic `User` role shouldn't be permitted to call E_1, or they should be permitted to call both E_1 and E_2.

More formally, an inconsistency exists if a role $u \in \mathcal{R}$ is permitted to access an endpoint $E_a \in \mathcal{E}$ that performs the same or a superset of the operations on a data entity as another endpoint $E_d \in \mathcal{E}$, to which u is denied access. Thus, given $\mathcal{A}_{e,d}$ is the set of CRUD operations $e \in \mathcal{E}$ performs on $d \in \mathcal{D}$, there is no inconsistency between $E_a \in \mathcal{E}$ and $E_d \in \mathcal{E}$ for data entity $D \in \mathcal{D}$ iff $\texttt{A_operations}(\mathcal{A}_{E_a,D})$ &$\texttt{A_operations}(\mathcal{A}_{E_d,D}) \neq \texttt{A_operations}(\mathcal{A}_{E_d,D})$. This **holds because**, if not all the bits in $\texttt{A_operations}(\mathcal{A}_{E_d,D})$ are in $\texttt{A_operations}(\mathcal{A}_{E_a,D})$, **then** E_d performs at least one type of data operation on D that E_a *does not perform*.

However, we also identify several instances where this scenario does not constitute a true inconsistency. Therefore, our methodology incorporates these exceptions into the constraint set generation. First, consider the case where the endpoint E_d doesn't allow u to call it as it makes calls to endpoints that u cannot call. This provides a valid justification for why u is not allowed to call E_d. To address this, we perform a breadth-first traversal to identify such cases and refrain from enforcing the constraint if the condition is met. Next, consider the case where endpoint E_d performs operations on another data entity that u should not be able to perform. This provides another legitimate reason why u is not allowed to call E_d. To address this, we enforce the constraint iff, for each set of operations that E_d performs on various data entities, E_a performs the same or a superset of the operations on the corresponding data entities. This leads to the formal definition of a data entity operation inconsistency:

Definition 2 *(Data Entity Operation Inconsistencies).* All data entity operation inconsistencies in $\mathcal{S}_{authorization}$ are contained by the set:

$$\mathcal{I}_{entities} = \left\{ \begin{array}{l} (u, E_a, E_d, D) \mid \\ \quad u \in \mathcal{R} \wedge E_a \in \mathcal{E} \wedge E_d \in \mathcal{E} \wedge D \in \mathcal{D} \\ \quad \wedge\, operations(E_a, D) \supseteq operations(E_d, D) \\ \quad \wedge\, \neg other_entities(E_a, E_d, D) \\ \quad \wedge\, calls_permitted(u, E_d) \\ \quad \wedge\, operations(E_a, D) \neq \emptyset \wedge operations(E_d, D) \neq \emptyset \\ \quad \wedge\, u \in \texttt{E_permitted}(E_a) \wedge u \notin \texttt{E_permitted}(E_d) \end{array} \right\}$$

where:

$operations(e, d) = \texttt{A_operations}(\texttt{E_accesses}(e, d))$. This returns the set of operations performed by e on d (or $\emptyset$ if no operation is performed).

$other_entities(E_a, E_d, d) \equiv \forall (D_{other} \neq d) \in \mathcal{D}$, $(operations(E_a, D_{other}) \neq \emptyset \wedge operations(E_d, D_{other}) \neq \emptyset) \Rightarrow (operations(E_a, D_{other}) \not\supseteq operations(E_d, D_{other}))$

This handles the edge case where an endpoint E_d performs operations on another data entity that u should not be able to perform. $calls_permitted(u, E_d) \equiv \forall E_{other} \in reach(E_d),\ (u \in \texttt{E_permitted}(E_{other}))$. This handles the edge case where an endpoint E_d doesn't permit u as there exists an endpoint E_{other} that E_d calls for which u is not permitted.

The BFS of endpoint call chains from E_d is $\text{reach}(E_d) = \{E_x \in \mathcal{E} : E_d \rightsquigarrow E_x,\ E_d E_x \in \mathcal{G}\}$ where $reach(E_d) \subseteq \mathcal{E}$.

$E_d \rightsquigarrow E_x \in \mathcal{G} \Leftrightarrow \exists\, k \geq 1,\ \exists E_{i_1}, \ldots, E_{i_k}\ E_{i_k} \in \mathcal{E}$ such that $(E_d, E_{i_1}) \in \mathcal{G} \wedge (E_{i_1}, E_{i_2}) \in \mathcal{G} \wedge \cdots \wedge (E_{i_k}, E_x) \in \mathcal{G}$.

The goal is to ensure $\mathcal{I}_{entities} = \emptyset$ Thus, we define a constraint to identify the inconsistencies represented in this set. By factoring in the exceptions and applying this constraint for every role and for every pair of endpoint E_a that permits u and endpoint E_d that denies u to perform operations in D, the entirety of the microservice system is validated by this constraint. This constraint is formally defined in Eq. 3 below.

$$C_{entities} = \bigwedge_{u \in R} \bigwedge_{E_a, E_d \in E} \bigwedge_{d \in D} A[E_a][d] \,\&\, A[E_d][d] \neq A[E_d][d] \vee other_entities(E_a, E_d, d) \vee \neg calls_permitted(u, E_d) \vee A[E_a][d] = 0 \vee A[E_d][d] = 0 \vee EP[E_a] \,\&\, u = 0 \vee EP[E_d] \,\&\, u \neq 0 \quad (3)$$

where:

$other_entities(E_a, E_d, d)$ is defined the same as in Definition 2 but with one difference. SMT-compatible constructs are used where $operations(e, d)$ is replaced by $A[e][d]$.

$calls_permitted(u, E_d)$ is defined the same as in Definition 2 but with one difference. SMT-compatible constructs are used where `E_permitted`(e) is replaced by $EP[e]$ and $(e_1, e_2) \in \mathcal{G}$ is replaced by $G[e_1][e_2]$.

3.2.3 At least one permitted role constraints

For each endpoint, we assert that at least one role must be permitted to access the endpoint. Otherwise, there is no reason for the endpoint to exist as no one is authorized to call it. For example, in the LibMS, if the `POST /loans` endpoint permitted no roles, then no one would be able to be loaned a book. The main justification for enforcing this constraint is to prevent the solver from denying all roles access to an endpoint. This can be formally defined as follows:

Definition 3 *(At Least One Permitted Role Inconsistency).* All at least one permitted role inconsistencies in $\mathcal{S}_{authorization}$ are contained by the set:

$$\mathcal{I}_{roles} = \{E \in \mathcal{E} \mid \texttt{E_permitted}(E) = \emptyset\}$$

The goal is to ensure $\mathcal{I}_{roles} = \emptyset$. Thus, we define a constraint to identify the inconsistencies represented in this set. This constraint is formally defined in Eq. 4 below.

$$C_{roles} = \bigwedge_{p_i \in EP} p_i > 0 \quad (4)$$

3.3 Applying solvers for authorization consistency verification

Once an SMT constraint model is generated, any type of SMT solver can be used to determine whether the constraint model is satisfiable, indicating whether the microservice system authorization policy is consistent (see ? in Fig. 2).

If the SMT constraint model is not satisfiable, an optimizer can be applied to produce a set of authorized roles for each endpoint that satisfies the constraint model (see ? in Fig. 2). The optimizer's goal is to limit the number of changes such that the set of authorized roles is as close as possible to those implemented in the actual system. If no solution exists, this means changes must be made to the system architecture or other aspects outside the variable authorized roles the solver is permitted to solve for.

4 Formal reasoning and proof

To establish the correctness of our methodology, we must demonstrate two key properties: (1) the IM accurately represents the microservice system, and (2) the SMT constraint model correctly enforces authorization consistency. This section provides formal reasoning and proofs for both properties, ensuring our approach delivers sound verification results.

4.1 On the correctness of the intermediate model

Our IM extends the framework established in [12], which demonstrates the feasibility and correctness of the IM through a model to system traceability analysis and a worked out example. We apply the same approach to validate our extended model $\mathcal{S}_{authorization}$.

First, Table 2 illustrates how aspects of our IM, $S_{authorization}$, can be extracted from microservice system source code repositories. We identify some potential limitations. Namely, static analysis is not foolproof; it may miss system aspects that make the IM inaccurate. For example, there may be issues classifying data operations based solely on repository functions or annotations. Additionally, aspect-oriented programming approaches or any other techniques that make the call graph difficult to predict could lead to data entity accesses by endpoints not being identified. Finally, static analysis tools may have to parse code written in specific frameworks, programming languages, or libraries separately due to the different paradigms and design patterns they employ [39].

However, Table 2 demonstrates that it is possible to extract each aspect of the IM from the source code. Since this extraction can be used with any language using an abstract syntax tree, it should be possible to develop similar static analyzers for each language, framework, etc. Only one tool should be required per language, framework, or library. The Python FastAPI, Java Spring Boot, and ASP.NET Core columns of Table 2 illustrate how the system model could feasibly be extracted from a selection of languages/frameworks. This means that our model is feasible to extract from source code and that it is representative of the actual microservice system.

Table 2 Model to System Traceability

Model component	Extracted from	Python fastAPI [40, 41]	Java spring boot [42, 43]	ASP.NET core [44]
$\mathcal{M}$: Microservices	Project structure, manifest/build files, and application entrypoints	`pyproject.toml`	`pom.xml`, `@SpringBootApplication`	`.csproj`
$\mathcal{E}$: Endpoints	REST controller definitions and route annotations/ decorators	`@app.get`, `@app.post`	`@GetMapping`, `@PostMapping`	`[HttpGet]`, `[HttpPost]`
`E_parent` : Endpoint Parents	Location of endpoint definitions within a microservice	Route function module	Controller class	Controller definition
$\mathcal{G}$: Call Graph Edges	AST/call graph traversal identifying framework-specific remote service calls	`requests.get()`, `httpx.get()`	`RestTemplate`, `WebClient`	`HttpClient.GetAsync()`
$\mathcal{D}$: Data Entities	ORM model definitions and entity annotations	SQLAlchemy `Base` models	`@Entity`	Entity Framework Core entity classes
$\mathcal{A}$: Data Entity Accesses	Repository/ORM function calls acting on entities	`session.add()`, `query()`, `delete()`	Repository methods	`DbContext.Add()`, `Find()`, `Remove()`
$\mathcal{O}$: Data Operations	User-defined operation categories (e.g., CRUD)	Derived from ORM method names	Derived from repository method names	Derived from `DbContext` method names
`A_operations` : Data Entity Access Operations	Names of repository/ORM functions	`add`, `query`, `delete`	`save`, `find`, `delete`	`Add`, `Find`, `Update`, `Remove`
$\mathcal{R}$: System Roles	Security configuration, policy definitions, and authorization annotations	Security dependencies (`Depends`)	Spring Security configuration	`[Authorize(Roles=...)]`
`E_permitted` : Endpoint Permitted Roles	Endpoint authorization annotations/ dependencies	Role-checking dependencies and middleware	Spring Security configuration	`[Authorize(Roles=...)]`
`E_accesses` : Endpoint Data Entity Accesses	Call graph traversal from endpoint to repository/ORM access	Endpoint→SQLAlchemy calls	Endpoint→Repository	Endpoint→`DbContext`

To further illustrate the feasibility and correctness of our model, we model a representative subset of two microservices in the TrainTicket microservice system benchmark [42][1]. TrainTicket was selected from a dataset of microservice systems [45] for its size and focus on replicating an industry system. We focused on modeling $\mathcal{M}$, $\mathcal{E}$, $\mathcal{D}$, $\mathcal{A}$, $\mathcal{O}$, `A_operations`, $\mathcal{R}$, and `E_permitted`.

In the TrainTicket system, there are two roles — admin and user. In addition, our system categorizes unauthenticated users as a separate role, even though it is not technically a role. This means $\mathcal{R} = \{admin, user, unauthenticated\}$. This can also be represented as a bitstring with one bit per role.

Furthermore, we focus on create, read, update, and delete (CRUD) operations performed on data entities. Thus, $\mathcal{O} = \{create, read, update, delete\}$. This can also be represented as a bitstring with one bit per operation.

Consider the following two microservices from the TrainTicket system: `ts-payment-service` and `ts-admin-user-service`, which could be modeled as M_1 and M_2 respectively. In M_1, there are four endpoints — labeled as E_1 through E_4. In M_2 there are five endpoints — labeled as E_5 through E_9. The payment

[1] TrainTicket is available at https://github.com/FudanSELab/train-ticket. The code from the following commit was used: 313886e99befb94be6cd45f085c98e0019f59829.

service interacts with two data entities: Money and Payment, which are considered D_1 and D_2, respectively. The admin user service does not directly access any data entities. Table 3 illustrates what endpoints access what data entities and what operations they perform. For each endpoint in M_1, only admins and users are permitted. Thus, $\forall_{e\in\{E_1,E_2,E_3,E_4\}}$ E_permitted$(e) = \{admin, user\}$. For each endpoint in M_2, only admins are permitted. Thus, $\forall_{e\in\{E_5,E_6,E_7,E_8,E_9\}}$ E_permitted$(e) = \{admin\}$. This completes the construction of the example model from the system source code.

4.2 On the correctness of the SMT constraint model

Having established that the IM accurately represents microservice systems, we now prove that our SMT constraint model correctly enforces authorization consistency based on the inconsistencies defined in Sect. 3.2.1, 3.2.2, and 3.2.3 in Definitions 1-3. We provide formal proofs for each constraint type and demonstrate that their combination ensures comprehensive system-wide consistency across the types of inconsistencies we define in Definitions 1-3.

4.2.1 Endpoint permitted role constraint proof

To verify Eq. 2, we present the following proof:

Axiom 1 *(Model Equivalence Validation). By definition in Sect. 3.1.1 and 3.1.3, the theoretical system model $\mathcal{S}_{authorization}$ and its SMT-friendly representation are logically equivalent. Operations on sets in the system model map directly to operations on bitstrings and arrays in the SMT model.*

Corollary 1 *(Subset Relation). Following established context in Axiom 1, the bitwise constraint $EP[E_i]\,\&\,EP[E_j] = EP[E_i]$ is logically and mathematically equivalent to the set-theoretic subset relation* E_permitted$(E_i) \subseteq$ E_permitted(E_j) .

Lemma 1 *($C_{endpoints}$ Universal Assertion). By Corollary 1, the SAT resolution of $C_{endpoints}$ universally enforces* E_permitted$(E_i) \subseteq$ E_permitted(E_j) *across all operational pathways $(E_i, E_j) \in \mathcal{G}$.*

Theorem 1 *(Endpoint Permitted Role Consistency Model). Applying Lemma 1 directly to Definition 1, a satisfied $C_{endpoints}$ dictates that no condition can exist where* E_permitted$(E_i) \not\subseteq$ E_permitted(E_j). *Consequently, the set evaluates entirely to $\mathcal{I}_{endpoints} = \emptyset$. Therefore, $C_{endpoints}$ is satisfiable iff $\mathcal{I}_{endpoints} = \emptyset$, proving that no endpoint permitted role inconsistencies are present in the system when the constraint system is satisfied.*

4.2.2 Data entity operation constraint proof

To verify Eq. 3, we present the following proof:

Axiom 2 *(Bitwise Superset Evaluation). From Axiom 1, for any bitstrings $A[E_a][d]$ and $A[E_d][d]$ representing sets of CRUD operations, the bitwise evaluation $A[E_a][d]\,\&\,A[E_d][d] = A[E_d][d]$ holds if and only if $operations(E_a, d) \supseteq operations(E_d, d)$.*

Lemma 2 *(Superset Negation). By Axiom 2, the SMT constraint $A[E_a][d]\,\&\,A[E_d][d] \neq A[E_d][d]$ is logically equivalent to the system model condition $operations(E_a, d) \not\supseteq operations(E_d, d)$.*

Axiom 3 *(Empty Operations Evaluation). From Axiom 1, the SMT constraint $A[E_a][d] = 0 \vee A[E_d][d] = 0$ holds if and only if the system model evaluates $operations(E_a, d) = \emptyset \vee operations(E_d, d) = \emptyset$.*

Axiom 4 *(Role Permissions Evaluation). From Axiom 1, the SMT constraint $(EP[E_a]\,\&\,u = 0) \vee (EP[E_d]\,\&\,u \neq 0)$ holds if and only if $u \notin$* E_permitted$(E_a) \vee u \in$ E_permitted(E_d) *in the system model.*

Lemma 3 *(Tuple-Level Consistency). From the formal definition of $\mathcal{I}_{entities}$, a specific tuple (u, E_a, E_d, d) constitutes an inconsistency if and only if a specific conjunction of six conditions is met. By De Morgan's laws, Axioms 3 and 4, and Lemma 2, the inner disjunctive clause of $C_{entities}$ represents the exact logical negation of this conjunction. Therefore, if the inner clause of $C_{entities}$ evaluates to true for a given tuple, that tuple cannot belong to $\mathcal{I}_{entities}$.*

Table 3 Data Entity Accesses in `ts-payment-service`

Data entity access ($\mathcal{A}$)	Operations performed
A_1: `POST /api/v1/paymentservice/payment/money` accesses Money via AddMoneyRepository.	A_operations$(A_1) = \{create\}$
A_2: `GET /api/v1/paymentservice/payment` accesses Payment via PaymentRepository.	A_operations$(A_1) = \{read\}$
A_3: `POST /api/v1/paymentservice/payment` accesses Payment via PaymentRepository.	A_operations$(A_1) = \{create, read\}$

Proposition 1 *($C_{entities}$ Universal Enforcement). The constraint $C_{entities}$ evaluates to true if and only if its inner disjunctive clause holds for every possible combination of $u \in \mathcal{R}$, $E_a, E_d \in \mathcal{E}$, and $d \in \mathcal{D}$.*

Theorem 2 *(Data Entity Operation Consistency Model). From Proposition 1, when $C_{entities}$ is SAT, the inner clause holds for all tuples. From Lemma 3 and Definition 2, this guarantees that no tuple (u, E_a, E_d, d) can satisfy the conditions required to be in $\mathcal{I}_{entities}$. Therefore, $C_{entities}$ is satisfiable iff $\mathcal{I}_{entities} = \emptyset$, proving that the system contains no data entity operation inconsistencies when the constraint system is satisfied.*

4.2.3 At least one permitted role constraint proof

To verify Eq. 4, we present the following proof:

Axiom 5 *(No Permitted Roles Equivalence). Given the logical equivalence of the system models in Axiom 1, a bit-string $p_i \in EP$ corresponding to an endpoint $E_i \in \mathcal{E}$ evaluates to 0 if and only if its mapped set of permitted roles is empty, i.e., $p_i = 0 \iff \texttt{E_permitted}(E_i) = \emptyset$. Conversely, $p_i > 0 \iff \texttt{E_permitted}(E_i) \neq \emptyset$.*

Lemma 4 *(Constraint Satisfaction). When the constraint system is SAT, the logical conjunction C_{roles} holds true, structurally enforcing that $\forall p_i \in EP,\ p_i > 0$.*

Proposition 2 *(Non-Empty Role Mapping). Following Axiom 5 and Lemma 4, a SAT assignment to the SMT constraint model mathematically implies that every endpoint in the system model maps to a non-empty set of allowed user roles, such that $\forall E \in \mathcal{E}$, $\texttt{E_permitted}(E) \neq \emptyset$.*

Theorem 3 *(At Least One Permitted Role Model). By Definition 3, the set of all at least one permitted role inconsistencies in $\mathcal{S}_{\text{authorization}}$ is defined as $\mathcal{I}_{roles} = \{E \in \mathcal{E} \mid \texttt{E_permitted}(E) = \emptyset\}$. From Proposition 2, it holds that no endpoint satisfies this condition when C_{roles} is SAT. Therefore, C_{role} is satisfiable iff $\mathcal{I}_{roles} = \emptyset$, proving that the system contains no at least one permitted role inconsistencies when the constraint system is satisfied.*

4.2.4 Authorization consistency constraint model proof

To verify Eq. 1, we present the following proof:

Axiom 6 *(Constraint Combination). The $\wedge$ (AND) operator requires that a composite constraint model evaluates to true if and only if every constituent constraint is independently satisfied.*

Proposition 3 *(Global Satisfiability). Following Axiom 6, the overall model constraint C (Eq. 1), defined as $C_{endpoints} \wedge C_{entities} \wedge C_{roles}$, is satisfiable if and only if $C_{endpoints}$, $C_{entities}$, and C_{roles} are all simultaneously satisfiable.*

Theorem 4 *(Comprehensive Constraint Model). From Proposition 3 and Theorems 1–3, the composite constraint C is satisfiable if and only if $\mathcal{I}_{endpoints} = \emptyset$, $\mathcal{I}_{entities} = \emptyset$, and $\mathcal{I}_{roles} = \emptyset$. Consequently, when C is satisfiable, the union of all inconsistency sets is entirely empty ($\mathcal{I}_{endpoints} \cup \mathcal{I}_{entities} \cup \mathcal{I}_{roles} = \emptyset$).*

Theorem 4 thus establishes the correctness of our overall SMT constraint model in successfully enforcing consistency and eliminating all defined authorization inconsistencies across the microservice system model.

5 Case study

To assess the methodology, we implemented a Proof-of-Concept (PoC) tool tailored to Java Spring microservices[2]. This choice reflects the platform's common use for microservices in the industry and a standard for role-based endpoint authorization. The tool uses the system source code (i.e., from a decentralized codebase). To encode the IM, it derives an IR of the system and encodes the microservice system into an IM using the tool presented in [17]. Using this IM, the PoC generates our proposed constraint model and evaluates it using the Z3 solver [26].

To validate the methodology, we need a suitable microservice benchmark system. We considered benchmarks in an existing dataset [45] and aimed at a selection of a mid-size or large system that uses role-based access control. From a peer-reviewed repository of microservice projects [45], we selected the TrainTicket microservice system [42] for evaluation. Compared to the other projects, TrainTicket had more inter-service communication pathways and was one of the biggest projects identified, with 41 microservices total. Further, TrainTicket also has properly implemented authorization policy throughout its microservices and avoids the anti-pattern of utilizing centralized authorization [36] These attributes make TrainTicket a suitably complex system for evaluating our tool.

Our case study has two primary objectives. First, we evaluate the accuracy, precision, and recall of our IM encoding plus our solver output. Second, we examine the running time of our methodology to demonstrate its feasibility. We structure the case study into five stages: (1) evaluating the

[2] Our proof-of-concept can be found at https://github.com/cloudhubs/authorization-consistency-solver.

accuracy of encoding the TrainTicket system into an IM (Sect. 5.1), (2) evaluating the efficacy of our SMT constraint model when supplied to the Z3 solver (Sect. 5.2), (3) conducting a fault injection experiment to measure recall (Sect. 5.3), (4) comparing our results with existing work [2, 8] (Sect. 5.4), and (5) discussing the impacts and feasibility of our methodology (Sect. 5.5). For all experiments, we use the same version of TrainTicket as in [2][3].

5.1 On the reliability of the derived intermediate model

Before the efficacy of the solver system can be evaluated, we must demonstrate that the information required by the solver can be feasibly extracted from the system's source code. We implement the combined approach using both traversal of an IR (enriched version of an AST) and pattern matching as described in Sect. 3. Using an IR provides method-level granularity of the TrainTicket system, allowing us to traverse method and REST calls. To obtain what roles were authorized for what endpoints, we employed regular expressions and pattern matching on the portions of the source code defining these policies.

5.1.1 Ground truth establishment

For validation of the IM, we use the ground truth of the TrainTicket system. Much of the ground truth was taken from the work of Abdelfattah et al. [2], including: (1) the roles present in the system, (2) the entities and the operations each endpoint performs in each repository, and (3) System Dependency Graph (SDG) representing remote service calls between endpoints. However, we manually created the ground truth for roles authorized to call each endpoint.

Table 4 Extracted versus Ground Truth Intermediate Model Constructs

Intermediate model metrics	Results				
	IMs (Count)	Ground truth (Count)	Precision (%)	Recall (%)	Accuracy (%)
Microservices	41	41	100	100	100
Endpoints	241	241	100	100	100
Entities	19	18	94.74	100	-
Endpoints With Correct Roles	236	241	-	-	97.92
Entity Accesses with Correct CRUD Operations	133	142	-	-	93.66
REST Calls	175	206	-	-	84.95

[3] Commit a4ed2433b0b6ab6e0d60115fc19efecb2548c6cd of TrainTicket available at https://github.com/FudanSELab/train-ticket was used.

5.1.2 Microservice & endpoint extraction results

As shown in Table 4, every microservice and endpoint was extracted and included in the generated IM.

5.1.3 Data entity extraction

Each data entity was also included correctly in the model; however, an extra entity was included in the generated model, introducing a slight loss in entity detection precision. This error arose because TrainTicket has two microservices for orders — `ts-order-service` and `ts-other-order-service` — that use the same data repository. Our PoC for IM encoding is limited in that it considers data repositories rather than data entities. We use a heuristic that if two data repositories have the same name, they represent the data entity; however, this did not hold true for the order and other order services. Extracting the names of the data entity that a data repository primarily manages would likely resolve this issue.

5.1.4 Endpoint role extraction

Next, the IM generator correctly identified role requirements for endpoints **97.92%** of the time. Five endpoints had their roles incorrectly identified because the `antMatchers` function matching each endpoint to a role authorization policy was passed a variable endpoint URL rather than a constant string (i.e.,/api/v1/orderservice/order). These instances might be fixed by using the initial value of the variable containing the URL since authorization policy usually should not change at runtime.

5.1.5 CRUD operation extraction

Meanwhile, as illustrated in Table 4, **93.66%** of CRUD operations on data entities were correctly detected by the IM generator. In order to accurately categorize which operations were performed, we used a heuristic-based approach using the function names to identify the types of operations performed by an endpoint. Every misclassified CRUD repository access was due to the misclassification of update versus create operations. This misclassification arises from the fact that, in Java Spring Boot, the same function is used to both create and update in a data repository. To address this, we first scan each data repository operation for keywords to identify whether the operation on the entity is a create or update, read, or delete operation. The words *"create"*, *"add"*, *"insert"*, *"save"*, and *"update"* indicate a create or update operation. The words *"read"*, *"find"*, *"get"*, and *"query"* indicate a read operation. Finally, the words *"delete"* or *"remove"* indicate a delete operation. Next,

Table 5 Identified Endpoint Role Requirements for TrainTicket Authorization Consistency

#	Endpoint	Suggested change	Identified inconsistency
1	`POST /api/v1/preserveservice/preserve`	Do not permit the admin role	Endpoint Permitted Role Inconsistency (Existing)
2	`POST /api/v1/orderservice/order/admin`	Permit the user role and unauthenticated users	Data Entity Operation Inconsistency (Existing)
3	`POST /api/v1/preserveotherservice/preserveOther`	Do not permit the admin role	Endpoint Permitted Role Inconsistency (Existing)
4	`DELETE /api/v1/travelservice/trips/{?}`	Permit the user role and unauthenticated users	Data Entity Operation Inconsistency (Existing)
5	`DELETE /api/v1/users/{?}`	Permit the user role and unauthenticated users	Endpoint Permitted Role Inconsistency (Existing)
6	`PUT /api/v1/orderservice/order/admin`	Permit the user role and unauthenticated users	Data Entity Operation Inconsistency (Existing)
7	`POST /api/v1/orderservice/order`	Permit unauthenticated users	Data Entity Operation Inconsistency (Existing)
8	`GET /api/v1/users`	Permit the user role and unauthenticated users	Data Entity Operation Inconsistency (Existing)
9	`PUT /api/v1/travelservice/trips`	Permit the user role and unauthenticated users	Data Entity Operation Inconsistency (Existing)
10	`PUT /api/v1/orderservice/order`	Permit unauthenticated users	Data Entity Operation Inconsistency (Existing)
11	`GET /api/v1/adminbasicservice/adminbasic/contacts`	Do not permit unauthenticated users	Data Entity Operation Inconsistency (caused by row 1)

we use the IR to associate the HTTP verb (`GET, POST, PUT, DELETE`, etc.) of the endpoint performing an operation on the data entity to determine if the create/update operation is a create or an update operation. We label operations performed in endpoints with a `POST` verb to be create operations, and we label operations with any other HTTP verb as an update operation.

5.1.6 REST call extraction

Of all the REST calls, **84.95%** were correctly included in the IM. Initially, 51 calls went unidentified due to a variable/field being used to pass the URL to the request function rather than just the path itself. However, we mitigated this by using the IR to identify the initial value of class fields; while these fields could technically change, they typically remain constant and were used to make the code more readable. This allowed us to identify 19 additional calls. In the end, there were 32 calls still could not be identified, primarily because the paths were stored in local variables not included in the IR. Including these fields in the IR would allow for the identification of every REST call in the IM.

Overall, these results demonstrate that extracting the information necessary to validate microservice system authorization policy is feasible. Despite some errors in the extraction of the IM, refining aspects of the process — such as the incorporating local variables in the IR for improving identification of REST calls — could significantly reduce these errors.

5.2 SMT solver and constraint model evaluation

We generate an SMT constraint model based on a manually corrected, ground truth version of the TrainTicket IM and supply the constraint model to the Z3 solver to evaluate the constraint model. To improve explainability, we initially insert each constraint using constant values from the IM only. However, if the constraint is unsatisfiable with the constants provided, we replace the constants with Z3 variables so that the solver can solve for a set of permissions that makes the system consistent.

5.2.1 Solver results

Table 5 illustrates the output of the Z3 solver when provided the SMT constraint model. According to the solver, the TrainTicket system in its current state is inconsistent, but with the modifications suggested by the solver, the system can be brought into a consistent state. The provided modifications from the solver **may not be optimal** as the current PoC suggests a solution that minimizes the number of changes required. Thus, it does not take into account business rules or context. However, these modifications provide useful insight into where inconsistencies in the system may lie.

5.2.2 Analysis setup

We analyze the 11 endpoints with role permissions that need to be changed to make the microservice system consistent.

We analyzed endpoints that share a REST call connection or an access to the same data entity as any of the 11 endpoints to explain why each change was suggested by the solver. We note that [11] presented preliminary data on a similar experiment with minimal analysis, but with a PoC that could only add permitted roles to endpoints (not remove them), which produced different results.

The TrainTicket system has two roles — *admin* and *user*. The IM also has a so-called unauthenticated role which is not actually a role but is used by the solver system to indicate that an endpoint does not require authentication. We encoded endpoint role requirements into a bitstring consisting of three bits. $roles_{req} \,\&\, 0b100 = 1$ indicates that the admin role is allowed to access an endpoint. Similarly, $roles_{req} \,\&\, 0b010 = 1$ indicates that the user role is allowed to access an endpoint, and $roles_{req} \,\&\, 0b001 = 1$ indicates that unauthenticated users are allowed to access an endpoint. While this particular system has only three roles, there is no bound on the number of roles that can be considered.

5.2.3 Endpoint permitted role inconsistencies

We now analyze each *identified inconsistency* to determine if it is one of the valid inconsistencies described in Definitions 1-3 in Sect. 3.2.1, 3.2.2, and 3.2.3. First, referencing each affected endpoint by its assigned number in Table 5, Row 1 illustrates an endpoint role inconsistency. This is caused by the fact that row 1's endpoint, which permits only the admin and user roles, calls `GET /api/v1/assuranceservice/assurances/{?}/{?}`, which only permits the user role. Thus, the solver's solution is to prevent row 1's endpoint from permitting the admin role.

Meanwhile, row 3's endpoint permits both the admin and user roles. However, it makes a call to `GET /api/v1/assuranceservice/assurances/{?}/{?}`, which permits only the user role. This creates an endpoint permitted role inconsistency. Thus, the solver's solution is to prevent row 3's endpoint from permitting the admin role.

Considering row 5, `DELETE /api/v1/userservice/users/{?}` calls row 5's endpoint. However, this endpoint permits all roles and unauthenticated users whereas row 5's endpoint permits only admins. Thus, there is an endpoint permitted role inconsistency, and the solver's solution is to have row 5's endpoint permit all users.

Row 11 is also involved in an endpoint permitted role inconsistency. Row 11's endpoint permits all roles and unauthenticated users. However, it calls the endpoint `GET /api/v1/contactservice/contacts`, which only permits the admin and user roles. Thus, the solver suggests preventing row 10's endpoint from permitting unauthenticated users.

5.2.4 Data entity operation inconsistencies

Next, the endpoints `PUT /api/v1/orderservice/order`, `POST /api/v1/orderOtherService/orderOther`, and `POST /api/v1/orderOtherService/orderOther/admin` permit all roles and unauthenticated users to perform a create and read on orders in `OrderRepository`. However, the endpoints in rows 2, 7, and 10 perform the same operations but do not permit all roles and unauthenticated users. This creates multiple data entity operation inconsistencies that can be solved by permitting all users to access the endpoints in rows 2, 7, and 10.

Similarly, row 6 is part of a data entity operation inconsistency. Its endpoint performs a read and update on orders in `OrderRepository`. However, `GET /api/v1/orderservice/order/orderPay/{?}` performs the same operations, yet allows all roles and unauthenticated users. Thus, the solver's solution is to have row 6's endpoint permit all users.

These inconsistencies arise from a misconfiguration in the `orderOtherService`'s security policy that allows all roles and unauthenticated users to access all its endpoints. While this solution from the solver is consistent, it would be more optimal to ensure that the admin endpoints remain admin-only and that unauthenticated users aren't permitted for operations that only signed in users should perform. While the solver does not provide the optimal solution, it successfully highlights important data entity operation inconsistencies.

Considering row 4, `DELETE /api/v1/travel2service/trips/{?}` allows all roles and unauthenticated users to perform read and delete operations on trips in `TripRepository`. However, row 4's endpoint provides the same operations but limits access to just the admin role, presenting a data entity operation inconsistency. Thus, the solver's solution is to have row 4's endpoint permit all users.

Also involving the `TripRepository`, row 9's endpoint performs a read and update operation on trips in the repository. However, row 9 only permits the admin role whereas `PUT /api/v1/travel2service/trips` performs the same operations in the repository yet permits all roles and unauthenticated users. Thus, there is a data entity operation inconsistency, and the solver's suggestion is to ensure row 9's endpoint also permits all users.

Finally, row 8 was identified by the solver as being part of a data entity access inconsistency, suggesting that all users, regardless of whether they are authenticated, should be allowed to access the endpoint. This is because the `POST /api/v1/users/login` endpoint in the authentication service is accessible to all roles and performs the same

operations that the row 8 does. Therefore, the assertion is that either both endpoints should be accessible to both roles or neither endpoint should be accessible to both roles. This correctly identifies an inconsistency as described by Definition 2 (Data Entity Operation Inconsistencies).

5.2.5 Solver discussion

In summary, all of the detected inconsistencies were valid based on Definitions 1-3, and we were unable to identify any instances where an inconsistency we defined was missed. The performance of the system on the endpoint permitted role consistency constraints and data entity operation consistency constraints demonstrates the feasibility of our methodology. However, there are situations in which the inconsistencies we detect can be too strict or not representative of business authorization policy in some cases.

Consider the `POST /api/v1/users/login` data entity operation inconsistency. While this was correctly identified as being an inconsistency inline with Definition 2, we should consider this inconsistency a false positive as unauthenticated users and regular users likely should not be able to get all of the user information in the database. Therefore, it would likely actually be consistent to allow all users to log in but only allow admins to access all users' information.

Thus, identifying the data entity operation inconsistencies as defined may not necessarily align with all business design decisions. However, since the inconsistency can be too strict rather than too lax, identifying these inconsistencies provides at minimum the areas of the code that would need to be examined for authorization policy implementation issues. Thus, in spite of these false positives, our methodology still feasibly offers significant value.

5.3 Fault injection experiment

While the analysis presented in Sect. 5.2 illustrates the feasibility of applying our methodology and how it can identify inconsistencies, it does not demonstrate that we find all inconsistencies. Thus, we conduct an additional experiment utilizing fault injection and manual analysis to measure the accuracy, precision, and recall of our system.

5.3.1 Experimental setup

We start with the ground truth model of the TrainTicket system. To create a baseline model in which faults can be injected, we use the solver suggestions presented in Table 5 to update the model. We then generate 30 different versions of the model, varying the number and type of faults injected. We consider three different types of faults:

- Permitted Role Fault - a random endpoint's permitted roles are changed to a different, random set of roles.
- Data Operation Fault - a combination of a Permitted Role Fault on a random endpoint e and a randomized change in data operations that e performs on a data entity.
- System Connection Fault - a new connection is established between any two endpoints in different microservices.

We base the faults we inject into the system based on real-world mistakes that developers could make. First, consider the permitted role fault. This is representative of a situation in which a developer has added or removed permitted roles from an endpoint, but failed to realize the impact (e.g., endpoints calling the endpoint have now broke, or the endpoint is now too permissive and allows users to attempt to call an endpoint they should not be allowed to call.) Such an oversight could occur due to the rapid evolution of microservice systems [4], coupled with the complexity of analyzing upstream and downstream dependencies of endpoints in remote service call chains [10].

Second, consider the data operation fault. This is representative of a situation in which a developer has provided new data operation scope to new users, but failed to realize it may grant a role more permissive access to perform certain operations on a data entity than another existing endpoint. Again, such an oversight could occur due to rapid evolution [4], but also due to the complexity of analyzing different or distributed data operations on the same data entity across different microservices [10].

Third, consider the system connection fault. This is representative of a situation in which a developer is updating an endpoint A and needs additional functionality from another endpoint B, but fails to consider if the new remote service call introduces a situation in which users authorized to call endpoint A but not endpoint B now attempt to call endpoint B. Such an oversight can occur for the same reasons a permitted role fault could occur: microservice system evolution and complex remote service call chain analysis.

We note that not all faults will result in an inconsistency. For example, a Permitted Role Fault may change an endpoint's role to not match business rule requirements, yet it may not present an inconsistency if it does not have connections to other endpoints, etc. We evaluate the solver's ability to identify inconsistencies in this experiment.

We chose these three types of faults as they represent the most reasonable aspects of the model where a fault could occur. For the Data Operation Fault, a combination of the Permitted Role Fault and a data operation change rather than just changing the data operation was selected as most data operation changes by themselves do not introduce an

inconsistency. For the System Connection Faults, we opted to only add new connections as removing connections would not introduce faults based on our constraint definitions.

The thirty models generated explore different combinations of fault numbers and types. For every combination of fault type and fault count, three tests were generated. Namely, test cases were generated for a system with one permitted role fault injected, a system with two permitted role faults, and a system with three permitted role faults. This yielded 9 test cases total. A similar pattern was performed with the data operation faults and system connection faults. This yielded an additional 18 test cases for 27 in total. Finally, three test cases were generated where one of each fault type was injected into a system. This brought the number of test cases to 30. All of the code, data, and results for this experiment can be found at https://github.com/cloudhubs/authorization-consistency-solver.

The models were then loaded and fed to the SMT constraint model generator before being supplied to the solver for verification. We saved the IMs, the exact faults injected into the IMs, and the results of the solver for each test case. Then, two authors independently reviewed all thirty test cases to determine if any inconsistencies were missed or if false inconsistencies were identified. Results were compared in the presence of a third author, who settled any differences in the two authors' results.

5.3.2 Experiment results

Across all 30 test cases, we did not identify any instances where the solver missed a type of inconsistency we detect (endpoint permitted role, data entity operation, and at least one permitted role constraints). Further, we did not identify any instances where an inconsistency was falsely detected. This **100% accuracy, precision, and recall** in identifying inconsistencies is inline with the correctness we formally demonstrate in Sect. 4.

5.4 Comparison with existing work

In our capturing of the microservice system architecture in our IM, we achieve nearly 100% accuracy in identifying all of the microservices, endpoints, and entities in the microservice, similar to [2]. In addition, we successfully identified 43 of the 51 create/update ground truth CRUD operations – an 84.31% accuracy. While this is not directly comparable with [2] as they duplicate some CRUD operations via their enumeration techniques, it illustrates that our methodology for IM extraction and encoding is viable and, at minimum, comparable with other methodologies proposed in the literature.

In addition, we measured the execution time of our PoC on a system with 32 GB of memory and an 11th-generation Intel i9-11980HK CPU with 8 cores. We timed the execution by running the PoC on this computer using the Train-Ticket benchmark. For the SMT constraint model generation and solving stages, the ground truth IM was used. We used ten samples to calculate each average. The average time to generate the IR takes **4.52±0.04 seconds** (excluding time to download the source code from remote repositories). Meanwhile, the IM encoding takes **0.45±0.19 seconds** on average to complete. The final phase of generating constraints and having the solver determine satisfiability took **1.23±0.006 seconds** on average. Thus, the overall process took a total of **6.20±0.048 seconds** on average.

We compare these results to the execution times of various testing tools such as Minikube, Docker CE, and Gremlin as reported in [8]. To run five test cases each five times on a benchmark with only 10 microservices (approximately 1/4 the size of TrainTicket with 241 endpoints across 41 microservices), these tools require about 5–6 seconds. Further, our 6.20 second execution time falls well within developer's psychological tolerance for CI and build tools, which has been established to be about 10 minutes [46, 47]. Hence, we demonstrate that our formal verification delivers both comprehensive coverage and practical performance.

However, this must be taken in the context of the trade-off between static analysis and runtime testing tools. Considering that we cover the entirety of a microservice system benchmark, our methodology shows clear potential for attaining extensive system coverage with competitive speed and less computational power requirements to other testing methods. However, it is crucial to note that this speed comes at the cost of capturing some of the additional semantics that are captured by runtime testing. Thus, our methodology would be optimal for parts of CI/CD process that require quick testing or low compute for rapid feedback for developers, whereas runtime testing would be optimal for evaluating aspects of the system not achievable via static analysis.

5.5 Discussion and implications

Our proof of concept proves the feasibility of our methodology and its capability to identify authorization consistency errors on a third-party microservice system. These errors would not be easily revealed by conventional tactics such as code reviews or testing. Our methodology is capable of verifying all combinations of interconnection paths. At the same time, it operates on an IM derived by static code analysis, which is a best-effort approximation of the SDG interconnection. While dynamic analysis could be considered for SDG, it faces identical challenges and can only be as reliable as the testing dataset is comprehensive.

Considering decentralized environments, where distinct teams manage distinct microservices, the likelihood of identifying authorization consistency errors across microservices during development is minimal given that development teams are unaware of the setting across multiple interconnected microservices. Our methodology can be integrated into development pipelines or Continuous Integration and Continuous Delivery (CI/CD) systems, serving as a reliable safeguard for decentralized development teams that can greatly reduce errors in critical infrastructure, limit ripple effects, and reduce costs incurred via root cause analysis and costly patches or rollbacks.

Consider scenarios where flights are delayed or canceled because boarding passes cannot be validated as a result of breaking change that introduces an authorization error. Or consider a situation where a customer using a credit card at a gas station has their transaction declined because of authorization errors resulting from misconfigured microservice communication. Such examples illustrate the implications of inconsistent authorization policy.

To the best of our knowledge, there is no commercial or scientific tool for the introduced problem, and our methodology provides an effective approach to address it by taking advantage of formal methods.

While our case study demonstrates the methodology on a single platform, microservices are developed on component-based development frameworks that use similar practices in Java/C#/C++/Python/Go, etc. All these frameworks recognize endpoints, controllers, services, repositories, and data entities, and all apply role-based access control. Hence, our approach should naturally extends to these platforms; however, additional case studies will be needed in the future to verify this as there could be limitations.

Our approach has the ability to suggest changes to authorized roles per endpoint that would make the authorization policy consistent. However, these changes do not take into account business context. For example, consider two endpoints: Endpoint A, where an admin could perform a READ on a data entity, and Endpoint B, where an admin and user can perform a READ on the same data entity. The data entity operation consistency constraint would require either allowing the user role to access Endpoint A or denying the user access to Endpoint B.

However, other than the other endpoint permitted role constraints and the at least one permitted role constraints, the solver has no information on which is the right choice. The only objective the solver has is to minimize the number of changes. Thus, while our approach is a method powerful for identifying issues and considering potential solutions, in real-world scenarios, it could be limited in its suggestions by the lack of business requirements and context. This further highlights how a combination of applying formal methods for rapid feedback & low compute plus performing periodic runtime testing for more runtime semantically aware results may provide the best results.

Our approach does not have upper bounds on the number of roles or interconnections in the system. However, there are some potential limitations related to the definition of the model and its extraction. For example, our methodology does not take into account the scope of the data that data operations are performed on (as highlighted in Sect. 5.2). Furthermore, other simplifications such as asserting that a system connection from an endpoint A to an endpoint B always results in a call every time endpoint A is called could also lead to false positives.

These limitations could be addressed by adding additional attributes to the model and including them in the constraints. For example, adding information on the scope of data attained (e.g., whether a single record or a list of records is being obtained) and traversing the AST further down from function calls to conditional calls would allow for more nuanced constraints and fewer false positives. This information could even potentially be used to extend the system model without needing to modify the constraints (e.g., the set $\mathcal{O}$ could potentially be modified to have bit representing "create one row", "create multiple rows", "read one row", "read multiple rows", etc. However, it remains to be seen with what accuracy some of this information can be extracted from system source code. Nonetheless, these limitations tend to introduce false positives rather than false negatives, which is critical for security policy as it means issues are unlikely to be missed.

6 Threats to validity

Following the framework established by Wohlin et al. [48], we analyze four categories of threats to validity: (1) construct validity, (2) internal validity, (3) external validity, and (4) conclusion validity. We address each category systematically to establish confidence in our methodology and findings. While our case study demonstrates promising results on a realistic benchmark system, we acknowledge several limitations that suggest directions for future work and broader validation.

6.1 Construct validity

Construct validity concerns whether our methodology and case study accurately represent real-world microservice authorization verification scenarios.

Benchmark Selection. To ensure that our methodology and case study are reflective of the real-world, we made sure to use a microservice system benchmark commonly used in

microservice system research. The TrainTicket system [42] was designed with replicating industrial systems in mind, making it suitable for simulating a real-world microservice system. In addition, we design our proof of concept to work with commonly used frameworks [19] that align with common standards [20, 21]. However, evaluation on additional microservice system benchmarks in the future with different frameworks or languages would help further strengthen the evaluation of the methodology. While our theoretical and logical analysis of our methodology shows no issue with generalization across systems with different frameworks or languages, our current case study focuses on Java Spring Boot, which cannot empirically prove the generality of our methodology.

Scope of Authorization Models. We consider CRUD operations performed with REST endpoints, which we assume have role-based access control and propagate user tokens between microservices. While all of these are common in the real-world [2, 38, 42], not all microservice systems use these standards. First, systems may employ specific trust boundaries or privilege elevations during a remote service call chain, meaning the endpoint permitted role constraints would raise false positives during the elevation. Second, systems employing attribute-based access control (ABAC), policy-based access control, or custom authorization frameworks may require extensions to our model in order to address concepts such as data ownership, scope of operations performed, and other authorization policy context.

Further, our case study considers a microservice system with only three roles. However, real-world and enterprise systems are likely to have numerous roles. We identify this as a limitation of our case study but note that based on the definition of the intermediate model and our formal reasoning, it is reasonable to presume that the methodology can support any number of roles since the bitstrings representing the roles can be of arbitrary length without affecting the theoretical correctness of the methodology. For example, a bitstring for a system with five roles (e.g., 0b00001 = unauthenticated, 0b00010 = user, 0b00100 = organization manager, 0b01000 = organization admin, and 0b10000 = super-admin), an endpoint permitted roles bitstring could look like 0b11100 to allow only organization managers, organization admins, and super-admins only.

Model Completeness. There are scenarios where our methodology is not fully comprehensive at all layers in the microservice system. Other aspects or layers of the endpoint, such as the control flow of the controllers and business logic, the scope of the data being obtained, or the data transfer objects the endpoint uses, could allow for more nuanced detection of inconsistencies and increased accuracy. Additional aspects of the overall system, such as nested data entities, replicated data, and role hierarchies, are additional complexities not currently included in our methodology that could be considered to improve our work.

Limitations of Static Analysis. While static analysis is a powerful tool in that it can extract much of the structure and constructs of a codebase, static analysis has difficulty in capturing the semantics of dynamic control flows, message queues, and other code features that are only apparent when running the codebase [14, 16, 17, 49]. This presents a challenge as our methodology is only as effective as the model that can be extracted from the source code. However, we note that there are other approaches in microservices static analysis literature that rely on static inspection of ASTs and source code [2, 15–17], making this a challenge beyond our methodology. Additional research directions, such as using LLMs to identify additional semantics from source code that would not normally be captured by static analysis, could also improve the robustness of the static approach [50]. Nonetheless, further research and analysis on the robustness of current static analysis tools would aid in identifying the efficacy of the foundational tools our system uses on additional microservice system benchmarks and enterprise systems.

Explainability. Finally, the explainability of the inconsistency could use improvement to better reduce developer burden during actual use. Multiple approaches and systems for explaining why a constraint model is unsatisfiable already exist and could potentially be applied to improve the explainability of the results of the solver [51].

6.2 Internal validity

Internal validity concerns whether confounding factors or errors in our data collection process influence our results.

Ground Truth Accuracy. Some threats to validity could arise from errors in our IM data. To prevent errors, we use and verify the previously verified ground truth provided with [2]. For the endpoint role permissions, which was not a part of the ground truth in [2], we manually construct and validate the ground truth from scratch to ensure its accuracy.

Manual Analysis Bias. For the fault injection experiment, we mitigate bias through independent review by two authors, with a third author resolving any disagreements. This triangulation approach reduces the risk of subjective interpretation affecting our conclusions.

6.3 External validity

External validity concerns whether our findings generalize beyond the specific case study and implementation choices.

Platform Specificity. One important threat to external validity is that we implement our proof of concept

specifically to parse a Java Spring Boot-based microservice system. We mitigate this by basing most of the generation of our IM off of IRs, which are "language-agnostic and adaptable to various project structures" [17, p. 10]. The endpoint authorization information that we don't obtain from the IR is obtained through pattern matching, which could likely be obtained in other frameworks or languages through pattern matching or parsing an abstract syntax tree.

Single System Evaluation. There is no indication from our work and case study that our methodology would not be generalizable; however, additional validation on other microservice systems is needed to fully ensure the effectiveness of our methodology. Future work should evaluate our approach on systems built with different frameworks (e.g., Node.js,.NET, Go), architectural patterns, and authorization models to establish broader applicability.

Fault Injection Representation. While we substantiate our choices for the faults we inject such that they would reasonably represent potential issues developers would face due to the complexities and unique nature of microservice systems, there is no standard benchmark or empirical study on what types of authorization faults are made in microservice systems. For example, there is no true way to determine the percent of the types of faults our methodology currently addresses. We mitigate this by making our faults representative of real-world concerns [10] and designing the model and constraint system to be extensible such that additional constraints could be added to identify different faults [12], providing additional utility and flexibility to our contributions.

6.4 Conclusion validity

Conclusion validity concerns whether our conclusions are appropriately supported by our empirical results.

Benchmark Availability. There is no standardized benchmark specific to verifying microservice authorization policy consistency due the novelty of our solution. To reduce threats to conclusion validity, we compare our work with similar work such as [2] where possible. To reduce bias and error in the fault injection study, two authors independently analyzed the data, with a third author serving as a tie breaker to reconcile any differences in each author's conclusions.

Bias Mitigation. To reduce error and bias in the fault injection study, two authors independently analyzed the data, with a third author serving as a tiebreaker to reconcile any differences in each author's conclusions. This process ensures that our findings are not influenced by individual researcher bias.

Statistical Significance. Our fault injection experiment with 30 test cases provides reasonable coverage of fault types and combinations. However, larger-scale studies with diverse systems would strengthen confidence in our precision and recall measurements.

Fault Injection Experimental Scope and Interpretation. Since we achieved perfect accuracy when performing fault injection, we note the bounds of our experiment as follows:

1. There could be inconsistencies present beyond the scope of what our model cover; they were not considered because such inconsistencies are not well defined, and
2. Our results do not imply that the solver's proposed changes to resolve the detected inconsistencies are optimal; rather, they indicate that all all inconsistencies were correctly identified; the proposed changes were excluded from the accuracy assessment due to our focus on identifying inconsistencies and the lack of business context to be able to make informed changes; incorporating either of these aspects could impact the accuracy.

In spite of these bounds, the experiments in Sect. 5.1, 5.2, and 5.3 illustrate promising correctness of our methodology, stemming from the application of formal methods.

7 Conclusions

This work presents a novel methodology for utilizing a representation of a microservice system to generate SMT constraints that verify the consistency of authorization policy across a microservice system when supplied to an SMT solver. Our methodology avoids manual inspection by automating comprehensive analysis of the entire system, reducing time to delivery while ensuring security integrity. Our findings demonstrate profound impact potential for microservices and cloud-native systems through new safeguards that reduce developer burden and prevent runtime security vulnerabilities.

We developed our methodology in the context of continuous formal verification by creating our intermediate models from intermediate representations, which can be incrementally updated [17]. The generic nature of using a Satisfiability Modulo Theories constraint model and solver allows for a diverse set of constraints to be generated. Our case study validates the approach on a 41-microservice benchmark, demonstrating high information extraction accuracy (84.95–100%), strong precision in validating authorization policy consistency (90.91%), and practical performance (6.20 seconds end-to-end). Thus, we establish the feasibility of formal verification for real-world microservice systems.

Future work could take advantage of said incremental updates to provide an efficient method for much-needed continuous verification of microservice system authorization policy [52]. Such work would require incremental addition and removal of not only elements of the IM, but also the constraints in the constraint system. Further, additional consideration would have to be taken on if the solver could be optimized to reuse work between increments to reduce solving time.

Future work could extend the model and constraints with additional properties of endpoints or other layers to address the limitations revealed in our case study. Specifically, incorporating fine-grained data access scopes, conditional call patterns, and role hierarchies would reduce false positives while maintaining recall. Our work also prepares a precedent for using SMT solvers and formal methods for other problems in microservice evolution research. Additional validation across diverse platforms (Node.js,.NET, Go) and authorization models (ABAC, policy-based) would establish broader applicability.

To the best of our knowledge, no existing commercial or academic tool addresses automated formal verification of authorization policy consistency at the application layer for microservice systems. As microservice adoption accelerates, formal verification approaches as ours are crucial safeguards in the software development lifecycle, ensuring that decentralized development does not compromise system-wide security integrity.

Acknowledgements We thank Dr. Amr Abdelfattah for his feedback and support on drafts of this manuscript.

Author contributions C.W. and T.C. developed the initial theories for the premise of the methodology. C.W., S.W., and T.C. wrote the main manuscript text. All figures were prepared by C.W. Tables were prepared by C.W., S.W., and T.C. C.W. built the proof of concept and collected the data for the case study. Analysis of the case study data was performed by C.W., S.W., and N.R. All authors provided feedback on the manuscript through development and reviewed the manuscript.

Funding This work was supported by the National Science Foundation under Grant No. 2409933. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Data availability For the benefit of the community and to encourage the reproduction of our work, we share the code and data used to conduct our case study. Link: https://github.com/cloudhubs/authorization-consistency-solver.

Declarations

Competing interests The authors declare no competing interests.

References

1. Michael Nieles, V.P. Kelley Dempsey: NIST SP 800-12 Rev. 1, An introduction to information security. NIST (2017). https://doi.org/10.6028/NIST.SP.800-12r1
2. Abdelfattah, A., Schiewe, M., Curtis, J., Cerny, T., Song, E.: Towards security-aware microservices: On extracting endpoint data access operations to determine access rights:. In: Proceedings of the 13th International Conference on Cloud Computing and Services Science, pp. 15–23. SCITEPRESS - Science and Technology Publications, Portugal (2023). https://doi.org/10.5220/0011707500003488
3. Amoroso d'Aragona, D., Li, X., Cerny, T., Janes, A., Lenarduzzi, V., Taibi, D.: One microservice per developer: Is this the trend in OSS? In: Papadopoulos, G.A., Rademacher, F., Soldani, J. (eds.) Service-Oriented and Cloud Computing, vol. vol. 14183, pp. 19–34. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-46235-1_2
4. Lenarduzzi, V., Besker, T., Taibi, D., Martini, A., Arcelli Fontana, F.: A systematic literature review on technical debt prioritization: Strategies, processes, factors, and tools. J. Syst. Softw. **171**, 110827 (2021). https://doi.org/10.1016/j.jss.2020.110827
5. Bogner, J., Fritzsch, J., Wagner, S., Zimmermann, A.: Industry practices and challenges for the evolvability assurance of microservices: An interview study and systematic grey literature review. Empirical Softw. Eng. **26**(5), 104 (2021). https://doi.org/10.1007/s10664-021-09999-9
6. Abdelfattah, A.S., Cerny, T., Yero, J., Song, E., Taibi, D.: Test coverage in microservice systems: An automated approach to E2E and API test coverage metrics. Electronics **13**(10), 1913 (2024). https://doi.org/10.3390/electronics13101913
7. Smith, S., Robinson, E., Frederiksen, T., Stevens, T., Cerny, T., Bures, M., Taibi, D.: Benchmarks for end-to-end microservices testing. In: 2023 IEEE International Conference on Service-Oriented System Engineering (SOSE), 60–66. IEEE, NJ, USA (2023). https://doi.org/10.1109/SOSE58276.2023.00013
8. Sotomayor, J.P., Allala, S.C., Alt, P., Phillips, J., King, T.M., Clarke, P.J.: Comparison of runtime testing tools for microservices. In: 2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC), **2**, 356–361 (2019). https://doi.org/10.1109/COMPSAC.2019.10232
9. Commission, F.C.: In the Matter of T-Mobile US, Inc. (DA-24-860A1). Accessed: 2026-06-13 (2024). https://docs.fcc.gov/public/attachments/DA-24-860A1.pdf
10. Černý, T., Donahoo, M., Trnka, M.: Contextual understanding of microservice architecture: current and future directions. ACM SIGAPP Applied Computing Review **17**, 29–45 (2018). https://doi.org/10.1145/3183628.3183631
11. Wojtak, C.: Formal methods for verifying authorization policy in microservice systems. In: 2025 IEEE International Conference on Service-Oriented System Engineering (SOSE), 241–242. IEEE, NJ, USA (2025). https://doi.org/10.1109/SOSE67019.2025.00033
12. Wojtak, C., Gajewski, D., Cerny, T.: Vision: An extensible methodology for formal software verification in microservice systems. In: 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS), pp. 274–280 (2025). https://doi.org/10.1109/MODELS67397.2025.00033
13. Hui, M., Wang, L., Li, H., Yang, R., Song, Y., Zhuang, H., Cui, D., Li, Q.: Unveiling the microservices testing methods, challenges, solutions, and solutions gaps: A systematic mapping study. J. Syst. Softw. **220**, 112232 (2025). https://doi.org/10.1016/j.jss.2024.112232

14. Zhang, W., Bali, D., Kerney, J., Panda, A., Shenker, S.: Extracting Database Access-control Policies From Web Applications. arXiv. Version Number: 2. https://doi.org/10.48550/ARXIV.2411.11380
15. Genfer, P., Zdun, U.: Exploring architectural evolution in microservice systems using repository mining techniques and static code analysis. In: Galster, M., Scandurra, P., Mikkonen, T., Oliveira Antonino, P., Nakagawa, E.Y., Navarro, E. (eds.) Software Architecture, vol. vol. 14889, pp. 157–173. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-70797-1_10
16. Cerny, T., Abdelfattah, A.S., Yero, J., Taibi, D.: From static code analysis to visual models of microservice architecture. Clust. Comput. **27**(4), 4145–4170 (2024). https://doi.org/10.1007/s10586-024-04394-7
17. Cerny, T., Goulis, G., Abdelfattah, A.S.: Towards change impact analysis in microservices-based system evolution. In: 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 159–169. IEEE, NJ, USA (2025). https://doi.org/10.1109/SANER64311.2025.00023
18. CloudFlare: Global, integrated API protection and monitoring. Accessed: 2024-12-01. https://www.cloudflare.com/lp/dg/product/api-security/
19. Spring: Authorize HttpServletRequests: Spring Security. Accessed: 2025-10-29. https://docs.spring.io/spring-security/reference/servlet/authorization/authorize-http-requests.html
20. Mordani, R.: JSR 250: Common Annotations for the Java^{TM} Platform (2016). https://jcp.org/en/jsr/detail?id=250
21. Hopkins, W.: JSR 375: JavaTM EE Security API (2009). https://jcp.org/en/jsr/detail?id=375
22. Bánáti, A., Kail, E., Karóczkai, K., Kozlovszky, M.: Authentication and authorization orchestrator for microservice-based software architectures. In: 41st International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), 1180–1184. (2018). https://doi.org/10.23919/MIPRO.2018.8400214
23. Spring: Method Security: Spring Security. Accessed: 2025-10-29. https://docs.spring.io/spring-security/reference/servlet/authorization/method-security.html
24. Britikov, K., Zlatkin, I., Fedyukovich, G., Alt, L., Sharygina, N.: SolTG: A CHC-based solidity test case generator. In: Gurfinkel, A., Ganesh, V. (eds.) Computer Aided Verification, vol. vol. 14681, pp. 466–479. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-65627-9_23
25. Ringer, T., Grossman, D., Schwartz-Narbonne, D., Tasiran, S.: A solver-aided language for test input generation. Proc. ACM Program. Lang. (2017). https://doi.org/10.1145/3133915
26. Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Ramakrishnan, C.R., Rehof, J. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, pp. 337–340. Springer, Berlin, Heidelberg (2008)
27. Barbosa, H., Barrett, C., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A versatile and industrial-strength SMT solver. In: Fisman, D., Rosu, G. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, pp. 415–442. Springer, Cham (2022)
28. Lattuada, A., Hance, T., Bosamiya, J., Brun, M., Cho, C., LeBlanc, H., Srinivasan, P., Achermann, R., Chajed, T., Hawblitzel, C., Howell, J., Lorch, J.R., Padon, O., Parno, B.: Verus: A practical foundation for systems verification. In: Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles. SOSP '24, pp. 438–454. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3694715.3695952
29. Cau, A., Janicke, H., Moszkowski, B.: Verification and enforcement of access control policies. Form. Methods Syst. Des. **43**(3), 450–492 (2013). https://doi.org/10.1007/s10703-013-0187-3
30. Armando, A., Ranise, S.: Automated Symbolic Analysis of ARBAC-Policies (Extended Version). arXiv.Version Number, 1. (2010). https://doi.org/10.48550/ARXIV.1012.5590
31. Sabri, K.E.: Automated verification of role-based access control policies constraints using Prover9. International Journal of Security, Privacy and Trust Management **4**(1), 01–10 (2015). https://doi.org/10.5121/ijsptm.2015.4101
32. Backes, J., Bolignano, P., Cook, B., Dodge, C., Gacek, A., Luckow, K., Rungta, N., Tkachuk, O., Varming, C.: Semantic-based automated reasoning for AWS access policies using SMT. In: 2018 Formal Methods in Computer Aided Design (FMCAD), 1–9. IEEE, NJ, USA (2018). https://doi.org/10.23919/FMCAD.2018.8602994
33. Amazon: How AWS uses automated reasoning to help you achieve security at scale. Accessed: 2025-04-12 (2018). https://aws.amazon.com/blogs/security/protect-sensitive-data-in-the-cloud-with-automated-reasoning-zelkova/
34. OSO: Authorization that takes you upmarket. Accessed: 2024-12-01. https://www.osohq.com/
35. OSO: Modeling in Polar - Authorization Modeling. Accessed: 2025-04-12. https://www.osohq.com/docs/modeling-in-polar
36. Cerny, T., Abdelfattah, A.S., Maruf, A.A., Janes, A., Taibi, D.: Catalog and detection techniques of microservice anti-patterns and bad smells: A tertiary study. J. Syst. Softw. **206**, 111829 (2023). https://doi.org/10.1016/j.jss.2023.111829
37. Lee, A.J., Winslett, M.: Safety and consistency in policy-based authorization systems. In: Proceedings of the 13th ACM Conference on Computer and Communications Security. CCS '06, pp. 124–133. Association for Computing Machinery, New York, NY, USA (2006). https://doi.org/10.1145/1180405.1180422
38. Almeida, M.G., Canedo, E.D.: Authentication and authorization in microservices architecture: A systematic literature review. Appl. Sci. **12**(6), 3023 (2022). https://doi.org/10.3390/app12063023
39. Dinh-Tuan, H., Mora-Martinez, M., Beierle, F., Garzon, S.R.: Development frameworks for microservice-based applications: Evaluation and comparison. In: Proceedings of the 2020 European Symposium on Software Engineering. ESSE '20, pp. 12–20. Association for Computing Machinery, New York, NY, USA (2020). https://doi.org/10.1145/3393822.3432339
40. FastAPI: FastAPI. Accessed: 2026-07-03. https://fastapi.tiangolo.com/
41. SQLAlchemy: SQLAlchemy Documentation. Accessed: 2026-07-03. https://docs.sqlalchemy.org/en/20/
42. Zhou, X., Peng, X., Xie, T., Sun, J., Xu, C., Ji, C., Zhao, W.: Benchmarking microservice systems for software engineering research. In: Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings. ICSE '18, 323–324. Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3183440.3194991
43. Spring: Documentation Overview: Spring Boot. Accessed: 2026-07-03. https://docs.spring.io/spring-boot/documentation.html
44. Microsoft: ASP.NET documentation. Accessed: 2026-07-03. https://learn.microsoft.com/en-us/aspnet/core
45. Amoroso d'Aragona, D., Bakhtin, A., Li, X., Su, R., Adams, L., Aponte, E., Boyle, F., Boyle, P., Koerner, R., Lee, J., Tian, F., Wang, Y., Nyyssölä, J., Quevedo, E., Rahaman, S.M., Abdelfattah, A.S., Mäntylä, M., Cerny, T., Taibi, D.: A dataset of microservices-based open-source projects. In: Proceedings of the 21st International Conference on Mining Software Repositories. MSR '24, 504–509. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3643991.3644890

46. Hilton, M., Nelson, N., Tunnell, T., Marinov, D., Dig, D.: Trade-offs in continuous integration: assurance, security, and flexibility. In: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. ESEC/FSE 2017, pp. 197–207. Association for Computing Machinery, New York, NY, USA (2017). https://doi.org/10.1145/3106237.3106270
47. Cornejo, O., Briola, D., Micucci, D., Ginelli, D., Mariani, L., Santos Parrilla, A., Juristo, N.: A family of experiments about how developers perceive delayed system response time. Software Qual. J. **32**(2), 567–605 (2024). https://doi.org/10.1007/s11219-024-09660-w
48. Wohlin, C., Runeson, P., Höst, M., Ohlsson, M., Regnell, B., Wesslén, A.: Experimentation in Software Engineering. Springer, Germany (2012). https://doi.org/10.1007/978-3-642-29044-2
49. Al Maruf, A., Bakhtin, A., Cerny, T., Taibi, D.: Using microservice telemetry data for system dynamic analysis. In: 2022 IEEE International Conference on Service-Oriented System Engineering (SOSE), pp. 29–38 (2022). https://doi.org/10.1109/SOSE55356.2022.00010
50. Wang, C., Zhang, W., Su, Z., Xu, X., Xie, X., Zhang, X.: Llmdfa: analyzing dataflow in code with large language models. In: Proceedings of the 38th International Conference on Neural Information Processing Systems. NIPS '24. Curran Associates Inc., Red Hook, NY, USA (2024)
51. Dev Gupta, S., Genc, B., O'Sullivan, B.: Explanation in constraint satisfaction: A survey. In: Zhou, Z.-H. (ed.) Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21, pp. 4400–4407. International Joint Conferences on Artificial Intelligence Organization, CA, USA (2021). https://doi.org/10.24963/ijcai.2021/601
52. Cook, B.: Formal reasoning about the security of amazon web services. In: Chockler, H., Weissenbacher, G. (eds.) Computer Aided Verification, vol. 10981, pp. 38–47. Springer, Cham, Switzerland (2018). https://doi.org/10.1007/978-3-319-96145-3_3

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.